

```

// ██████████ < START > ██████████
// FILE NAME: : view.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/view.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████ < START > ██████████
// FILE-PATH: packages/core/tests/view.test.ts
//

/**
 * @file packages/core/tests/view.test.ts
 * @description Unit tests for View base class state construction, base rendering, category styling, and identity helper.
 */

import { FallbackTheme } from '@pendex/theme';
import { describe, expect, test } from 'bun:test';
import {
  identity,
  View,
  type Category,
  type Config,
  type State,
  type StyleFunction,
} from '../src';

class TestView extends View {
  public getState(): State {
    return this.state;
  }

  public callCategoryStyle(
    category: Category,
    fallback: StyleFunction,
  ): StyleFunction {
    return this.categoryStyle(category, fallback);
  }
}

const baseConfig: Config = {
  theme: {
    name: 'pendex',
    path: './themes',
  },
  outputDir: 'OUT',
  rebuiltDir: 'REBUILT',
  exclude: [],
  jobs: [],
};

const categoryColors = { source: '#799470', web: '#6F92B0' };

describe('View Base Class', (): void => {
  test('identity generic helper returns value unchanged', (): void => {
    expect(identity('hello')).toBe('hello');
    expect(identity(42)).toBe(42);
  });

  test('View stores state as given, including categoryColors', (): void => {
    const view = new TestView({
      theme: FallbackTheme,
      config: baseConfig,
      categoryColors,
    });
  });
}

```

```

        expect(view.getState().categoryColors).toBe(categoryColors);
    });

    test('View stores state as given when categoryColors is absent', (): void => {
        const view = new TestView({ theme: FallbackTheme, config: baseConfig });
        expect(view.getState().categoryColors).toBeUndefined();
    });

    test('base View.prototype.render returns undefined gracefully', async (): Promise<void> => {
        const view = new View({ theme: FallbackTheme, config: baseConfig });
        const result = await view.render();
        expect(result).toBeUndefined();
    });

    test('categoryStyle returns brand hex style when defined and fallback when absent', (): void => {
        const view = new TestView({
            theme: FallbackTheme,
            config: baseConfig,
            categoryColors,
        });
        const fallbackFn: StyleFunction = (text: string): string =>
            `FALLBACK:${text}`;

        // Category with color defined ('source' -> '#799470')
        const sourceStyler = view.callCategoryStyle('source', fallbackFn);
        const sourceRes = sourceStyler('code');
        expect(sourceRes).toContain('code');
        expect(sourceRes).not.toContain('FALLBACK:');

        // Category without color defined ('style')
        const styleStyler = view.callCategoryStyle('style', fallbackFn);
        const styleRes = styleStyler('code');
        expect(styleRes).toBe('FALLBACK:code');
    });
});

// =====< END >-=====
// FILE NAME: : view.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/view.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : constants.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/constants.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/core/tests/constants.test.ts
//

/**
 * @file packages/core/tests/constants.test.ts
 * @description Unit tests for path string helpers and the app-wide Constants singleton.
 */

import { describe, expect, test } from 'bun:test';
import { baseName, Constants, dirName, extName, joinPath } from '../src';

describe('Constants and Path Helpers', (): void => {
    describe('joinPath', (): void => {
        test('should join segments with forward slashes', (): void => {

```

```

        expect(joinPath('a', 'b', 'c')).toBe('a/b/c');
    });

    test('should collapse multiple slashes', (): void => {
        expect(joinPath('a/', '/b//', 'c')).toBe('a/b/c');
    });

    test('should handle empty segments', (): void => {
        expect(joinPath('a', '', 'b', 'c')).toBe('a/b/c');
    });
});

describe('baseName', (): void => {
    test('returns the final segment of a posix path', (): void => {
        expect(baseName('src/utils/helper.ts')).toBe('helper.ts');
    });

    test('handles backslash separators', (): void => {
        expect(baseName('src\\utils\\helper.ts')).toBe('helper.ts');
    });

    test('returns the input when there is no separator', (): void => {
        expect(baseName('helper.ts')).toBe('helper.ts');
    });
});

describe('extName', (): void => {
    test('returns the extension with leading dot', (): void => {
        expect(extName('src/index.ts')).toBe('.ts');
    });

    test('returns only the last extension', (): void => {
        expect(extName('archive.tar.gz')).toBe('.gz');
    });

    test('returns empty string when there is no extension', (): void => {
        expect(extName('Makefile')).toBe('');
    });

    test('returns empty string for dotfiles (dot at index 0)', (): void => {
        expect(extName('.gitignore')).toBe('');
    });
});

describe('dirName', (): void => {
    test('returns the directory portion', (): void => {
        expect(dirName('src/utils/helper.ts')).toBe('src/utils');
    });

    test('handles backslash separators', (): void => {
        expect(dirName('src\\utils\\helper.ts')).toBe('src/utils');
    });

    test('returns "." when there is no separator', (): void => {
        expect(dirName('helper.ts')).toBe('.');
    });
});

describe('Constants object', (): void => {
    const divider: string = '████████████████████████████████████████';

    test('exposes the expected app-wide values', (): void => {
        const constantsAccess = Constants as unknown as {

```

```

        BLOCK: string;
        WIDTH: number;
    };
    expect(constantsAccess.BLOCK).toBe('');
    expect(constantsAccess.WIDTH).toBe(40);

    expect(Constants.OUTPUT_DIR).toBe('ALL');
    expect(Constants.REBUILT_DIR).toBe('ALL_REBUILT');
    expect(Constants.ENCODING).toBe('utf-8');
    expect(Constants.DIVIDER).toBe(divider);
    expect(Constants.GITIGNORE_PATH.endsWith('.gitignore')).toBe(true);
    expect(
        Constants.RUNTIME_CONFIG_PATH.endsWith('runtime.config.json'),
    ).toBe(true);
    });
});

// =====< END >-=====
// FILE NAME: : constants.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/constants.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : filescanner.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/filescanner.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/core/tests/filescanner.test.ts
//

/**
 * @file packages/core/tests/filescanner.test.ts
 * @description Unit tests for `FileScanner.ts` glob resolution, ignore pattern loading, and empty directory detection.
 */

import { afterAll, beforeAll, describe, expect, test } from 'bun:test';
import { mkdirSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';
import {
    findEmptyDirectories,
    loadIgnorePatterns,
    resolveJobFiles,
    type Job,
} from '../src';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
    PACKAGE_TESTS_DIR,
    `test-sandbox-filescanner-${process.pid}`,
);
const ORIGINAL_CWD: string = process.cwd();

const makeJob = (include: string[], exclude: string[] = []): Job => ({
    filename: 'x.txt',
    category: 'source',
    description: 'test job',
    include,
    exclude,
});

```

```

describe('FileScanner', (): void => {
  beforeAll(async (): Promise<void> => {
    await createSandbox(SANDBOX_DIR);
    process.chdir(SANDBOX_DIR);

    mkdirSync('src', { recursive: true });
    writeFileSync('src/a.ts', 'a');
    writeFileSync('src/b.ts', 'b');
    writeFileSync('notes.md', 'notes');
    writeFileSync('skip.log', 'log');
  });

  afterAll(async (): Promise<void> => {
    process.chdir(ORIGINAL_CWD);
    await cleanupSandbox(SANDBOX_DIR);
  });

  describe('loadIgnorePatterns', (): void => {
    test('returns [] when the file does not exist', async (): Promise<void> => {
      const patterns = await loadIgnorePatterns('./no-such-ignore-file');
      expect(patterns).toEqual([]);
    });

    test('strips comments and blank lines', async (): Promise<void> => {
      writeFileSync(
        '.testignore',
        `# comment\n\nnode_modules/\ndist/\n  \n`,
      );
      const patterns = await loadIgnorePatterns('.testignore');
      expect(patterns).toEqual(['node_modules/', 'dist/']);
    });
  });

  describe('resolveJobFiles —normal jobs', (): void => {
    test('matches include globs minus excludes', async (): Promise<void> => {
      const files = await resolveJobFiles(
        makeJob(['src/**/*.ts'], ['**/b.ts']),
        ['**/b.ts'],
        new Set(),
      );
      expect(files.map(f => f.replace(/\\/g, '/))).toEqual(['src/a.ts']);
    });

    test('deduplicates a file matched by multiple patterns', async (): Promise<void> => {
      const files = await resolveJobFiles(
        makeJob(['src/**/*.ts', '**/a.ts']),
        [],
        new Set(),
      );
      const posix = files.map(f => f.replace(/\\/g, '/'));
      expect(posix.filter(f => f === 'src/a.ts')).toHaveLength(1);
    });
  });

  describe('resolveJobFiles —remainder jobs (empty include)', (): void => {
    test('picks up everything not claimed and not excluded', async (): Promise<void> => {
      const claimed = new Set(['src/a.ts', 'src/b.ts']);
      const files = await resolveJobFiles(
        makeJob([],
          ['**/*.log'],
          claimed,
        ),
      );
    });
  });

```

```

    const posix = files.map(f => f.replace(/\\/g, '/')).sort();

    expect(posix).toContain('notes.md');
    expect(posix).not.toContain('src/a.ts');
    expect(posix).not.toContain('src/b.ts');
    expect(posix).not.toContain('skip.log');
  });

  test('with nothing claimed or excluded, sees all files', async (): Promise<void> => {
    const files = await resolveJobFiles(makeJob([], [], new Set()));
    expect(files.length).toBe(5);
  });
});

describe('findEmptyDirectories', (): void => {
  test('finds empty dirs and skips excluded ones', async (): Promise<void> => {
    mkdirSync('empty-one', { recursive: true });
    mkdirSync('excluded-empty', { recursive: true });
    mkdirSync('src/nested-empty', { recursive: true });

    const dirs = (
      await findEmptyDirectories('.', [
        '**/excluded-empty/**',
        '**/excluded-empty',
        'excluded-empty/**',
        'excluded-empty',
      ])
    )
      .map(d => d.replace(/\\/g, '/'))
      .sort();

    expect(dirs).toContain('empty-one');
    expect(dirs).toContain('src/nested-empty');
    expect(dirs.some(d => d.includes('excluded-empty'))).toBe(false);
    expect(dirs).not.toContain('src');
  });

  test('returns [] when no empty dirs exist', async (): Promise<void> => {
    const result = await findEmptyDirectories('src', []);
    expect(result.filter(d => !d.includes('nested-empty'))).toEqual([]);
  });
});
});

// ██████████< END >────────────────────────────────────────────────────────────────────────────────
// FILE NAME: : filescanner.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/filescanner.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< END >────────────────────────────────────────────────────────────────────────────────

// ██████████< START >────────────────────────────────────────────────────────────────────────────────
// FILE NAME: : bootstrap.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/bootstrap.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< START >────────────────────────────────────────────────────────────────────────────────
// FILE-PATH: packages/core/tests/bootstrap.test.ts
//
// oxlint-disable typescript/no-explicit-any
//
/**
 * @file packages/core/tests/bootstrap.test.ts

```

```

* @description Unit tests for bootstrap runner dependency resolution logic.
*/

import { Colors } from '@pendex/color';
import { ThemeManager } from '@pendex/theme';
import { afterAll, beforeAll, describe, expect, test } from 'bun:test';
import { writeFileSync } from 'node:fs';
import { join } from 'node:path';
import {
  type ResolvedDeps,
  ConfigManager,
  Constants,
  resolveRunnerDeps,
} from '../src';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
  PACKAGE_TESTS_DIR,
  `test-sandbox-bootstrap-${process.pid}`,
);
const ORIGINAL_CWD: string = process.cwd();

describe('bootstrap.resolveRunnerDeps', (): void => {
  let deps: ResolvedDeps;

  const loadDeps = async (themeName?: string): Promise<ResolvedDeps> => {
    if (themeName !== undefined) {
      writeFileSync(
        Constants.RUNTIME_CONFIG_PATH,
        JSON.stringify({ theme: themeName }),
      );
    }

    ConfigManager.resetInstance();
    ThemeManager.resetInstance();

    return await resolveRunnerDeps();
  };

  beforeAll(async (): Promise<void> => {
    await createSandbox(SANDBOX_DIR);
    process.chdir(SANDBOX_DIR);
    Colors.enable();
    ConfigManager.resetInstance();
    ThemeManager.resetInstance();
    deps = await resolveRunnerDeps();
  });

  afterAll(async (): Promise<void> => {
    process.chdir(ORIGINAL_CWD);
    ConfigManager.resetInstance();
    ThemeManager.resetInstance();
    await cleanupSandbox(SANDBOX_DIR);
  });

  test('1. resolves the shipped default theme by real file stem', async (): Promise<void> => {
    expect(deps.config.theme.name).toBe('pendex');
    expect(deps.theme.primary('x').toString()).toContain('38;2;214;164;72');
  });

  test('2. categoryColors is present for the shipped default config', async (): Promise<void> => {
    expect(deps.categoryColors).toBeDefined();
    expect(Object.keys(deps.categoryColors ?? {}).not.toHaveLength(0);
  });

```

```

});

test("3. categoryColors carries every Category the theme's [brand] table names exactly", async (): Promise<void> => {
  expect(deps.categoryColors?.source).toBeDefined();
  expect(deps.categoryColors?.web).toBeDefined();
  expect(deps.categoryColors?.style).toBeDefined();
  expect(deps.categoryColors?.terminal).toBeDefined();
  expect(deps.categoryColors?.configuration).toBeDefined();
  expect(deps.categoryColors?.documentation).toBeDefined();
  expect(deps.categoryColors?.testing).toBeDefined();
  expect(deps.categoryColors?.misc).toBeDefined();
});

test('4. pendex default exposes the expected source category color', async (): Promise<void> => {
  expect(deps.categoryColors?.source).toBe('#799470');
});

test('5. runtime override can switch to another shipped theme', async (): Promise<void> => {
  const overridden: ResolvedDeps = await loadDeps('dracula');

  expect(overridden.config.theme.name).toBe('pendex');
  expect(overridden.theme.primary('x').toString()).toContain(
    '\u001B[38;2;214;164;72m\u001B[39m',
  );
});

test('6. unknown theme names degrade safely and omit category colors', async (): Promise<void> => {
  const overridden: ResolvedDeps = await loadDeps('no-such-theme');

  expect(overridden.config.theme.name).toBe('pendex');
  expect(overridden.theme.primary('safe').toString()).toContain('safe');
  expect(overridden.categoryColors?.source).toEqual('#799470');
});
});

// =====< END >-=====
// FILE NAME: : bootstrap.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/bootstrap.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : archiveformat.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/archiveformat.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/core/tests/archiveformat.test.ts
//
import { describe, expect, test } from 'bun:test';
import {
  archivePathFor,
  buildArchiveEntry,
  joinArchiveEntries,
  parseArchive,
} from '../src';

describe('ArchiveFormat (frame-delimited, verbatim, no tearline)', () => {
  test('build → parse recovers path and content exactly', () => {
    const original = 'console.log("hello");';
    const results = parseArchive(
      buildArchiveEntry('src/index.ts', original),
    );
  });
});

```



```

    );

    expect(results).toHaveLength(1);
    expect(results[0]!.originalPath).toBe('src/index.ts');
    expect(results[0]!.content).toBe(original);
  });

  test('verbatim edge cases: trailing newlines, empty file, CRLF, leading whitespace', () => {
    for (const original of [
      'x\n',
      'x\n\n',
      '',
      '\n',
      'a\r\nb\r\n',
      '\n  indented\n',
    ]) {
      const results = parseArchive(
        buildArchiveEntry('src/f.ts', original),
      );
      expect(results).toHaveLength(1);
      expect(results[0]!.content).toBe(original);
    }
  });

  test('multiple entries round-trip without any separator between them', () => {
    const joined = joinArchiveEntries([
      buildArchiveEntry('src/a.ts', 'one\n'),
      buildArchiveEntry('src/b.ts', 'two'),
      buildArchiveEntry('src/c.ts', 'three\n'),
    ]);
    const results = parseArchive(joined);

    expect(results).toHaveLength(3);
    expect(results[0]!.content).toBe('one\n');
    expect(results[1]!.content).toBe('two');
    expect(results[2]!.originalPath).toBe('src/c.ts');
  });

  test('rebuilt content contains no banner residue', () => {
    const results = parseArchive(
      buildArchiveEntry('a/b.ts', 'const x = 1;'),
    );
    const content = results[0]!.content;

    expect(content).not.toContain('■');
    expect(content).not.toContain('/// PATH');
    expect(content).not.toContain('/// PROCESSED');
  });

  test('FULL self-hosting: an entire archive as content survives byte-perfect', () => {
    // The path-matched END frame is what makes this work: the inner
    // entries' frames carry inner paths, so they can never terminate
    // the outer entry. The old tearline format could not do this.
    const innerArchive = joinArchiveEntries([
      buildArchiveEntry('inner/x.ts', 'inner content\n'),
      buildArchiveEntry('inner/y.ts', 'more inner'),
    ]);
    const outer = joinArchiveEntries([
      buildArchiveEntry('OUT/code.txt', innerArchive),
      buildArchiveEntry('after.ts', 'still here'),
    ]);

    const results = parseArchive(outer);
  });

```

```

    expect(results).toHaveLength(2);
    expect(results[0]!.originalPath).toBe('OUT/code.txt');
    expect(results[0]!.content).toBe(innerArchive);
    expect(results[1]!.content).toBe('still here');
  });

  test('garbage between entries is skipped, never fatal', () => {
    const raw = `random junk\n${buildArchiveEntry('a.ts', 'one')}\nnoise\n${buildArchiveEntry('b.ts', 'two')}\ntrailing`;
    const results = parseArchive(raw);
    expect(results).toHaveLength(2);
    expect(results[0]!.content).toBe('one');
    expect(results[1]!.content).toBe('two');
  });

  test('an unterminated START frame is skipped without consuming later entries', () => {
    const startOnly = buildArchiveEntry('lost.ts', 'orphan')
      .split('\n')
      .slice(0, 8)
      .join('\n');
    const raw = `${startOnly}\n\n${buildArchiveEntry('ok.ts', 'fine')}`;
    const results = parseArchive(raw);
    expect(results).toHaveLength(1);
    expect(results[0]!.originalPath).toBe('ok.ts');
  });

  test('empty input and pure junk yield an empty result set', () => {
    expect(parseArchive('')).toHaveLength(0);
    expect(parseArchive('just some text')).toHaveLength(0);
  });

  test('a frame with a blank path is not treated as an entry', () => {
    const entry = buildArchiveEntry('src/x.ts', 'content');
    const lines = entry.split('\n');
    lines[3] = '// PATH      :      '; // start frame path blanked
    expect(parseArchive(lines.join('\n'))).toHaveLength(0);
  });

  test('windows-style path separators are normalized to posix', () => {
    const results = parseArchive(
      buildArchiveEntry('src\\utils\\helper.ts', 'x'),
    );
    expect(results[0]!.originalPath).toBe('src/utils/helper.ts');
  });

  test('archivePathFor joins output paths', () => {
    expect(archivePathFor('OUT', 'code.txt')).toBe('OUT/code.txt');
  });
});

// =====< END >-=====
// FILE NAME: : archiveformat.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/archiveformat.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : config.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/config.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/core/tests/config.test.ts

```

```

//
// oxlint-disable typescript/no-explicit-any
//
/**
 * @file packages/core/tests/config.test.ts
 * @description Unit tests for ConfigManager loading, runtime overrides, reload, and saving.
 */

import {
  afterAll,
  afterEach,
  beforeAll,
  beforeEach,
  describe,
  expect,
  mock,
  test,
} from 'bun:test';
import { existsSync, rmSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';
import { type Config, type Job, ConfigManager, Constants } from '../src';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
  PACKAGE_TESTS_DIR,
  `test-sandbox-config-${process.pid}`,
);
const ORIGINAL_CWD: string = process.cwd();

// Container for dynamic runtime mocks targeting internal file readers
let mockTomlResponse: any = null;

// Explicitly wrap the module mock invocation in void to satisfy oxlint
void mock.module('bun', () => {
  // Access the global namespace directly to bypass recursive import loops
  const nativeTomlParse = Bun.TOML.parse;

  return {
    // Export the global Bun context directly to retain local internal references
    ...Bun,
    TOML: {
      ...Bun.TOML,
      parse: (text: string): any => {
        // If a test declared an explicit mock payload, return it directly
        if (mockTomlResponse) {
          return mockTomlResponse;
        }
        return nativeTomlParse(text);
      },
    },
  };
});

function safeDelete(path: string): void {
  if (existsSync(path)) {
    try {
      rmSync(path, { force: true, recursive: true });
    } catch {
      try {
        writeFileSync(path, '', 'utf-8');
      } catch {
        /* ignore */
      }
    }
  }
}

```

```

    }
  }
}

describe('Configuration Logic (Config.ts)', (): void => {
  beforeAll(async (): Promise<void> => {
    await createSandbox(SANDBOX_DIR);
    process.chdir(SANDBOX_DIR);
  });

  afterAll(async (): Promise<void> => {
    process.chdir(ORIGINAL_CWD);
    ConfigManager.resetInstance();
    await cleanupSandbox(SANDBOX_DIR);
  });

  beforeEach(): void => {
    ConfigManager.resetInstance();
    mockTomlResponse = null;
    safeDelete(Constants.RUNTIME_CONFIG_PATH);
  });

  afterEach(): void => {
    ConfigManager.resetInstance();
    mockTomlResponse = null;
    safeDelete(Constants.RUNTIME_CONFIG_PATH);
  });

  test('getInstance loads default config correctly', async (): Promise<void> => {
    const manager: ConfigManager = await ConfigManager.getInstance();
    expect(manager.get()).toBeDefined();
  });

  test('getInstance returns the SAME instance on repeat calls (singleton)', async (): Promise<void> => {
    const first = await ConfigManager.getInstance();
    const second = await ConfigManager.getInstance();
    expect(second).toBe(first);
  });

  test('merges a runtime.config.json file over defaults', async (): Promise<void> => {
    writeFileSync(
      Constants.RUNTIME_CONFIG_PATH,
      JSON.stringify({
        theme: { name: 'dracula' },
        outputDir: 'CUSTOM_OUTPUT',
      }),
    );
    const manager = await ConfigManager.getInstance();
    expect(manager.get().outputDir).toBe('CUSTOM_OUTPUT');
  });

  test('falls back to defaults when runtime.config.json is corrupt', async (): Promise<void> => {
    writeFileSync(
      Constants.RUNTIME_CONFIG_PATH,
      'this is { not valid json',
    );
    const manager = await ConfigManager.getInstance();
    expect(manager.get()).toBeDefined();
  });

  test('reload re-reads and applies disk changes', async (): Promise<void> => {
    const manager = await ConfigManager.getInstance();
    writeFileSync(

```

```

        Constants.RUNTIME_CONFIG_PATH,
        JSON.stringify({ outputDir: 'RELOADED' })),
    );
    await manager.reload();
    expect(manager.get().outputDir).toBe('RELOADED');
});

test('save persists the in-memory config to runtime.config.json', async (): Promise<void> => {
    const manager = await ConfigManager.getInstance();
    const config = manager.get();
    (config as any).outputDir = 'SAVED_OUTPUT';

    await manager.save();

    const onDisk = await Bun.file(Constants.RUNTIME_CONFIG_PATH).json();
    expect(onDisk.outputDir).toBe('SAVED_OUTPUT');

    ConfigManager.resetInstance();
    const fresh = await ConfigManager.getInstance();
    expect(fresh.get().outputDir).toBe('SAVED_OUTPUT');
});

test('resetToDefaults discards runtime overrides in memory without touching disk', async (): Promise<void> => {
    writeFileSync(
        Constants.RUNTIME_CONFIG_PATH,
        JSON.stringify({ outputDir: 'OVERRIDDEN' })),
    );
    const manager = await ConfigManager.getInstance();
    expect(manager.get().outputDir).toBe('OVERRIDDEN');

    await manager.resetToDefaults();
    const onDisk = await Bun.file(Constants.RUNTIME_CONFIG_PATH).json();
    expect(onDisk.outputDir).toBe('OVERRIDDEN');
});

test('withDiskOverrides handles and recovers from runtime exclusion structural errors', async (): Promise<void> => {
    writeFileSync(
        Constants.RUNTIME_CONFIG_PATH,
        JSON.stringify({ exclude: ['custom-ignore-path'] })),
    );

    ConfigManager.resetInstance();
    const manager = await ConfigManager.getInstance();

    expect(manager.get().exclude).toContain('custom-ignore-path');
    expect(manager.get().exclude).toContain(Constants.RUNTIME_CONFIG_PATH);
});

test('recovers gracefully to base config if withDiskOverrides encounters critical file errors', async (): Promise<void> => {
    const manager = await ConfigManager.getInstance();
    writeFileSync(Constants.RUNTIME_CONFIG_PATH, '{"theme": ', 'utf-8');

    const currentConfig = await (ConfigManager as any).withDiskOverrides(
        manager.get(),
    );
    expect(currentConfig).toBeDefined();
});

test('handles fallback defaults and sparse fields for mapped jobs successfully inside the engine file', async (): Promise<void> => {
    // 1. Extract the private static method signature
    const readTomlDefaultsFn = (ConfigManager as any).readTomlDefaults;

    // 2. Build the targeted sparse TOML payload

```

```

const minimalisticToml = [
  'outputDir = "ALL"',
  'outputType = "txt"',
  'http = true',
  'exclude = []',
  '',
  '[[jobs]]',
  'filename = "engine-sparse-job.txt"', // Leaves out category, description, include, exclude
].join('\n');

// 3. Directly spy on and intercept Bun's file reader method for this execution
const originalBunFile = Bun.file;
(Bun as any).file = (path: string) => {
  // If the engine tries to open a configuration file, feed it our sparse in-memory string fake
  if (path.includes('config.toml') || path.includes('config/')) {
    return {
      text: async () => minimalisticToml,
      exists: async () => true,
    } as any;
  }
  return originalBunFile(path);
};

try {
  // 4. Run the production code directly (isolated from the singleton cache)
  const parsedConfigResult: Config = await readTomlDefaultsFn();

  // 5. Query the processed structure
  const targetJob = parsedConfigResult.jobs.find(
    (j: Job) => j.filename === 'engine-sparse-job.txt',
  );

  expect(targetJob).toBeDefined();
  // This explicitly validates the category ?? 'misc' fallback line inside Config.ts
  expect(targetJob?.category).toBe('misc');
  expect(targetJob?.description).toBe('');
  expect(targetJob?.include).toEqual([]);
  expect(targetJob?.exclude).toEqual([]);
} finally {
  // Restore Bun's built-in filesystem bindings safely so surrounding tests don't break
  (Bun as any).file = originalBunFile;
}
});

test('handles runtime config job mapping when overrides are structurally sparse inside the engine file', async (): Promise<void> => {
  ConfigManager.resetInstance();
  const manager = await ConfigManager.getInstance();

  // 2. Prepare a mock configuration override array item
  // We inject it into index position 99, ensuring baseJob evaluates as undefined!
  const sparseJobsOverride: any[] = [];
  sparseJobsOverride[99] = { filename: 'engine-override-sparse.txt' };

  // Save our custom parameters to the filesystem
  writeFileSync(
    Constants.RUNTIME_CONFIG_PATH,
    JSON.stringify({
      jobs: sparseJobsOverride,
    }),
  );

  const updatedConfig = await (ConfigManager as any).withDiskOverrides(
    manager.get(),
  );

```

```

    );
    const targetJob = updatedConfig.jobs.find(
      (j: any) => j.filename === 'engine-override-sparse.txt',
    );

    expect(targetJob).toBeDefined();
    // Confirms that baseJob missing evaluations resolve down to 'misc' inside Config.ts
    expect(targetJob?.category).toBe('misc');
    expect(targetJob?.exclude).toEqual([]);
  });

  test('assignKey utility copies defined properties and completely ignores undefined parameters', (): void => {
    // Import or use assignKey directly from your core modules hook
    const { assignKey: assignKeyFn } = require('../src');

    // Test Branch 1: Value is explicitly defined (forces execution of lines 91-93)
    const targetObject = { themeName: 'pendex', httpEnabled: true };
    const sourceWithUpdate = { themeName: 'dracula' };

    assignKeyFn(targetObject, sourceWithUpdate, 'themeName');
    expect(targetObject.themeName).toBe('dracula');

    // Test Branch 2: Value is explicitly undefined (forces bypass evaluation of lines 91-93)
    const sourceWithUndefined = { themeName: undefined };

    assignKeyFn(targetObject, sourceWithUndefined, 'themeName');
    // Verifies the target value was preserved untouched instead of overwriting with undefined
    expect(targetObject.themeName).toBe('dracula');
  });
});

// =====< END >-=====
// FILE NAME: : config.test.ts
// TYPE      : .TS
// PATH      : packages/core/tests/config.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : theme.test.ts
// TYPE      : .TS
// PATH      : packages/theme/tests/theme.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/theme/tests/theme.test.ts
//
import { Colors } from '@pendex/color';
import { beforeAll, describe, expect, test } from 'bun:test';
import { FALLBACK_THEME, FallbackTheme, useTheme } from '../src';

const ALL_STYLE_KEYS = [
  'primary',
  'secondary',
  'success',
  'warning',
  'error',
  'info',
  'muted',
  'title',
  'subtitle',
  'color',
  'backgroundColor',
  'bold',

```

```

    'italic',
    'textDecoration',
  ] as const;

describe('Theme System (chaining engine over the fallback brand theme)', (): void => {
  beforeAll((): void => Colors.enable());

  test('every style method renders the text', (): void => {
    for (const key of ALL_STYLE_KEYS) {
      expect(FallbackTheme[key]('test').toString()).toContain('test');
    }

    expect(FallbackTheme.textTransform('test').toString()).toBe('TEST');
  });

  test('styles chain (stacked before text arrives)', (): void => {
    expect(FallbackTheme.primary.bold('test').toString()).toContain('test');
  });

  test('callable syntax: text first, style after (eager application)', (): void => {
    expect(FallbackTheme('test').primary.toString()).toContain('test');
  });

  test('supports eager multi-style chaining after text exists', (): void => {
    const rendered = FallbackTheme('test').primary.bold.italic.toString();

    expect(rendered).toContain('test');
  });

  test('string coercion via template literal, String(), and valueOf()', (): void => {
    expect(`${FallbackTheme.bold('x')}`).toContain('x');
    expect(String(FallbackTheme.error('y'))).toContain('y');
    expect(FallbackTheme.info('z').valueOf()).toContain('z');
  });

  test('explicit toString returns the compiled value', (): void => {
    expect(FallbackTheme.warning('warn').toString()).toContain('warn');
  });

  test('calling with no text yields an empty renderable', (): void => {
    expect(`${FallbackTheme()}`).toBe('');
  });

  test('unknown property resolves to undefined (Proxy fall-through)', (): void => {
    const themeAsRecord = FallbackTheme as unknown as Record<
      string,
      unknown
    >;

    expect(themeAsRecord['definitelyNotAStyle']).toBeUndefined();
  });

  test('useTheme() can wrap FALLBACK_THEME directly and behaves like FallbackTheme export', (): void => {
    const localTheme = useTheme(FALLBACK_THEME);

    expect(localTheme.primary('same').toString()).toContain('same');
    expect(localTheme.textTransform('abc').toString()).toBe('ABC');
  });
});

// =====< END >=====
// FILE NAME: : theme.test.ts
// TYPE      : .TS

```



```
// PATH      : packages/theme/tests/theme.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// _____< END >_____

// _____< START >_____
// FILE NAME: : thememanager.test.ts
// TYPE      : .TS
// PATH      : packages/theme/tests/thememanager.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// _____< START >_____
// FILE-PATH: packages/theme/tests/thememanager.test.ts
//
import { Colors } from '@pendex/color';
import { afterEach, beforeEach, describe, expect, mock, test } from 'bun:test';
import {
    type DefaultTheme,
    type PendexTheme,
    type Theme,
    BRAND_PALETTE,
    ThemeManager,
} from '../src';

type BrandType = Record<string, string> | undefined;

describe('ThemeManager (TOML theme loading)', (): void => {
    beforeEach(() : void => Colors.enable());
    afterEach(() : void => {
        ThemeManager.resetInstance();
        mock.restore();
    });

    test('loads pendex and produces a working chainable Theme', async (): Promise<void> => {
        const manager: ThemeManager = await ThemeManager.getInstance('pendex');

        expect(manager.name()).toBe('pendex');

        const theme: Theme = manager.get();
        const styled: string = theme.primary('hello').toString();

        expect(styled).toContain('hello');

        // Brass #D6A448 = rgb(214, 164, 72)
        expect(styled).toContain('38;2;214;164;72');
    });

    test('getInstance() with no name defaults to pendex', async (): Promise<void> => {
        const manager: ThemeManager = await ThemeManager.getInstance();

        expect(manager.name()).toBe('pendex');
    });

    test('loads each shipped theme file', async (): Promise<void> => {
        const THEME_NAMES = [
            'pendex',
            'dracula',
            'tokyonight',
            'onedark',
            'monokaipro',
        ] as const;

        for (const name of THEME_NAMES) {
            ThemeManager.resetInstance();
        }
    });
});
```

```

        const manager: ThemeManager = await ThemeManager.getInstance(name);

        expect(manager.name()).toBe(name);
        expect(manager.get().success('ok').toString()).toContain('ok');
    }
});

test("extended() exposes pendex.toml's full schema, including tables buildTheme() never reads", async (): Promise<void> => {
    const manager: ThemeManager = await ThemeManager.getInstance('pendex');
    const extended: PendexTheme = manager.extended();

    expect(extended.name).toBe('Pendex');
    expect(extended.author).toBe('Dusty Dew');
    expect(extended.description).toContain('parchment');

    expect(extended.colors.primary).toBe('#D6A448');
    expect(extended.colors.background).toBe('#1A1A1A');
    expect(extended.colors.titleBg).toBe('#262626');

    expect(extended.palette?.brass).toBe('#D6A448');
    expect(extended.ansi?.black).toBeDefined();
    expect(extended.syntax?.keyword).toBeDefined();
    expect(extended.git?.added).toBeDefined();
    expect(extended.diagnostics?.hint).toBeDefined();
    expect(extended.diff?.addedBackground).toBeDefined();
    expect(extended.brand?.bookCover).toBeDefined();

    const CATEGORY_NAMES = [
        'source',
        'web',
        'style',
        'terminal',
        'configuration',
        'documentation',
        'testing',
        'misc',
    ] as const;

    for (const category of CATEGORY_NAMES) {
        expect(extended.brand?.[category]).toBeDefined();
    }
});

test('every shipped theme defines a complete [brand] table for category keys', async (): Promise<void> => {
    const THEME_NAMES = [
        'dracula',
        'tokyonight',
        'onedark',
        'monokaipro',
    ] as const;

    const CATEGORY_NAMES = [
        'source',
        'web',
        'style',
        'terminal',
        'configuration',
        'documentation',
        'testing',
        'misc',
    ] as const;

    for (const name of THEME_NAMES) {

```

```

    ThemeManager.resetInstance();

    const manager: ThemeManager = await ThemeManager.getInstance(name);
    const brand: BrandType = manager.extended().brand;

    for (const category of CATEGORY_NAMES) {
        expect(brand?.[category]).toBeDefined();
    }
}
});

test('extended() degrades to brand-only (no tables) for an unknown theme', async (): Promise<void> => {
    const manager: ThemeManager = await ThemeManager.getInstance(
        'no-such-theme-either',
    );
    const extended: PendexTheme = manager.extended();

    expect(extended.name).toBe('no-such-theme-either');
    expect(extended.colors).toEqual(BRAND_PALETTE);
    expect(extended.brand).toBeUndefined();
    expect(extended.ansi).toBeUndefined();
});

test('dracula theme uses the dracula purple for primary', async (): Promise<void> => {
    const manager: ThemeManager = await ThemeManager.getInstance('dracula');

    // #BD93F9 = rgb(189, 147, 249)
    expect(manager.get().primary('p').toString()).toContain(
        '38;2;189;147;249',
    );
});

test('an unknown theme name degrades to the brand palette, never throws', async (): Promise<void> => {
    const manager: ThemeManager =
        await ThemeManager.getInstance('no-such-theme');

    const styled: string = manager.get().primary('safe').toString();

    expect(styled).toContain('safe');
    expect(styled).toContain('38;2;214;164;72');
});

test('getInstance returns the same instance regardless of later names', async (): Promise<void> => {
    const first: ThemeManager = await ThemeManager.getInstance('dracula');
    const second: ThemeManager = await ThemeManager.getInstance('onedark');

    expect(second).toBe(first);
    expect(second.name()).toBe('dracula');
});

test('raw() exposes the flat DefaultTheme before proxy wrapping', async (): Promise<void> => {
    const manager: ThemeManager = await ThemeManager.getInstance('pendex');
    const raw: DefaultTheme = manager.raw();

    expect(raw.primary('raw')).toContain('raw');
    expect(raw.textTransform('abc')).toBe('ABC');
});

test('concurrent first calls share the same in-flight singleton promise', async (): Promise<void> => {
    const [a, b, c] = await Promise.all([
        ThemeManager.getInstance('pendex'),
        ThemeManager.getInstance('dracula'),
        ThemeManager.getInstance('onedark'),
    ]);
});

```

```

});

expect(a).toBe(b);
expect(b).toBe(c);
expect(['pendex', 'dracula', 'onedark']).toContain(a.name());
});

test('parse failure while reading a file degrades safely to brand fallback', async (): Promise<void> => {
  const originalFile = Bun.file;
  const originalParse = Bun.TOML.parse;

  await mock.module('bun', () => ({
    BunFile: class {},
  }));

  // Patch globals directly for Bun runtime behavior
  // @ts-expect-error test patch
  Bun.file = (() => ({
    exists: async () => true,
    text: async () => 'not actually valid toml',
  })) as typeof Bun.file;

  Bun.TOML.parse = (() => {
    throw new Error('parse failure');
  }) as typeof Bun.TOML.parse;

  try {
    const manager: ThemeManager =
      await ThemeManager.getInstance('broken-theme');

    expect(manager.name()).toBe('broken-theme');
    expect(manager.extended().colors).toEqual(BRAND_PALETTE);
    expect(manager.get().primary('safe').toString()).toContain('safe');
  } finally {
    Bun.file = originalFile;
    Bun.TOML.parse = originalParse;
  }
});

test('missing file path degrades safely to brand fallback', async (): Promise<void> => {
  const originalFile = Bun.file;

  // @ts-expect-error
  Bun.file = (() => ({
    exists: async () => false,
    text: async () => {
      throw new Error('should not be called');
    },
  })) as typeof Bun.file;

  try {
    const manager: ThemeManager =
      await ThemeManager.getInstance('missing-theme');

    expect(manager.name()).toBe('missing-theme');
    expect(manager.extended().colors).toEqual(BRAND_PALETTE);
  } finally {
    Bun.file = originalFile;
  }
});
});

```

```
// FILE NAME: : thememanager.test.ts
// TYPE      : .TS
// PATH      : packages/theme/tests/thememanager.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// _____< END >_____

// _____< START >_____
// FILE NAME: : themepalette.test.ts
// TYPE      : .TS
// PATH      : packages/theme/tests/themepalette.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// _____< START >_____
// FILE-PATH: packages/theme/tests/themepalette.test.ts
//
// Unit tests for ThemePalette's pure functions —no file I/O, so every
// edge case (partial files, garbage tables, wrong top-level types,
// normalization/fallback behavior, and buildTheme output) is exercised
// directly instead of through a real TOML file on disk.
// ThemeManager.test.ts covers the file-loading integration.

import { Colors } from '@pendex/color';
import { describe, expect, test } from 'bun:test';
import {
  type DefaultTheme,
  type PendexTheme,
  BRAND_PALETTE,
  FALLBACK_THEME,
  THEME_COLOR_KEYS,
  buildTheme,
  parsePendexTheme,
} from '../src';

Colors.enable();

describe('ThemePalette constants', (): void => {
  test('THEME_COLOR_KEYS defines the required root slots in order', (): void => {
    expect(THEME_COLOR_KEYS).toEqual([
      'primary',
      'secondary',
      'success',
      'warning',
      'error',
      'info',
      'muted',
      'foreground',
      'background',
      'titleBg',
    ]);
  });
});

test('BRAND_PALETTE provides every required color slot', (): void => {
  for (const key of THEME_COLOR_KEYS) {
    expect(BRAND_PALETTE[key]).toMatch(/^[0-9A-F]{6}$/);
  }
});

test('FALLBACK_THEME is a ready-to-use DefaultTheme', (): void => {
  const styled = FALLBACK_THEME.primary('fallback').toString();

  expect(styled).toContain('fallback');
  expect(typeof FALLBACK_THEME.bold).toBe('function');
  expect(typeof FALLBACK_THEME.textTransform).toBe('function');
});
```

```

});

describe('buildTheme', (): void => {
  test('returns a working DefaultTheme whose semantic color slots render text', (): void => {
    const theme: DefaultTheme = buildTheme(BRAND_PALETTE);

    expect(theme.primary('x')).toContain('x');
    expect(theme.secondary('x')).toContain('x');
    expect(theme.success('x')).toContain('x');
    expect(theme.warning('x')).toContain('x');
    expect(theme.error('x')).toContain('x');
    expect(theme.info('x')).toContain('x');
    expect(theme.muted('x')).toContain('x');
  });

  test('title, subtitle, color, and backgroundColor render text', (): void => {
    const theme: DefaultTheme = buildTheme(BRAND_PALETTE);

    expect(theme.title('Title')).toContain('Title');
    expect(theme.subtitle('Subtitle')).toContain('Subtitle');
    expect(theme.color('Body')).toContain('Body');
    expect(theme.backgroundColor('Bg')).toContain('Bg');
  });

  test('bold, italic, underline, and textTransform work', (): void => {
    const theme: DefaultTheme = buildTheme(BRAND_PALETTE);

    expect(theme.bold('bold')).toContain('bold');
    expect(theme.italic('italic')).toContain('italic');
    expect(theme.textDecoration('underline')).toContain('underline');
    expect(theme.textTransform('mixedCase')).toBe('MIXEDCASE');
  });

  test('uses palette colors in output ANSI sequences', (): void => {
    const theme: DefaultTheme = buildTheme(BRAND_PALETTE);

    const primary = theme.primary('p');
    const background = theme.backgroundColor('b');

    // Brass #D6A448 = rgb(214, 164, 72)
    expect(primary).toContain('38;2;214;164;72');

    // Background #1A1A1A = rgb(26, 26, 26)
    expect(background).toContain('48;2;26;26;26');
  });
});

describe('parsePendexTheme', (): void => {
  test('reads color scalars off the root, not a nested [colors] table', (): void => {
    const theme: PendexTheme = parsePendexTheme(
      {
        name: 'X',
        primary: '#111111',
      },
      'fallback',
    );

    expect(theme.colors.primary).toBe('#111111');
  });

  test('a theme overriding one color still gets a complete ThemeColors via brand fallback', (): void => {
    const theme: PendexTheme = parsePendexTheme(
      {

```

```

        primary: '#FF00FF',
    },
    'partial',
);

expect(theme.colors.primary).toBe('#FF00FF');
expect(theme.colors.secondary).toBe(BRAND_PALETTE.secondary);
expect(theme.colors.background).toBe(BRAND_PALETTE.background);
});

test('captures metadata fields when present', (): void => {
    const theme: PendexTheme = parsePendexTheme(
        {
            name: 'Pendex',
            author: 'Dusty Dew',
            version: '1.0.0',
            description: 'desc',
        },
        'fallback',
    );

    expect(theme.name).toBe('Pendex');
    expect(theme.author).toBe('Dusty Dew');
    expect(theme.version).toBe('1.0.0');
    expect(theme.description).toBe('desc');
});

test('missing metadata fields are undefined, not thrown', (): void => {
    const theme: PendexTheme = parsePendexTheme({}, 'fallback-name');

    expect(theme.name).toBe('fallback-name');
    expect(theme.author).toBeUndefined();
    expect(theme.version).toBeUndefined();
    expect(theme.description).toBeUndefined();
});

test('captures every extended table present in the file', (): void => {
    const theme: PendexTheme = parsePendexTheme(
        {
            palette: { brass: '#D6A448' },
            website: { link: '#3F5D78' },
            terminal: { cursor: '#D6A448' },
            ansi: { black: '#2C2C2C' },
            syntax: { keyword: '#C78C5C' },
            git: { added: '#728C69' },
            diagnostics: { hint: '#6Aafb5' },
            diff: { addedBackground: '#2A3A2A' },
            brand: { compile: '#D6A448' },
        },
        'x',
    );

    expect(theme.palette?.brass).toBe('#D6A448');
    expect(theme.website?.link).toBe('#3F5D78');
    expect(theme.terminal?.cursor).toBe('#D6A448');
    expect(theme.ansi?.black).toBe('#2C2C2C');
    expect(theme.syntax?.keyword).toBe('#C78C5C');
    expect(theme.git?.added).toBe('#728C69');
    expect(theme.diagnostics?.hint).toBe('#6Aafb5');
    expect(theme.diff?.addedBackground).toBe('#2A3A2A');
    expect(theme.brand?.compile).toBe('#D6A448');
});

```

```

test('an absent table is undefined, not an empty object', (): void => {
    const theme: PendexTheme = parsePendexTheme(
        {
            name: 'X',
        },
        'fallback',
    );

    expect(theme.brand).toBeUndefined();
    expect(theme.ansi).toBeUndefined();
});

test('an empty object table degrades to undefined', (): void => {
    const theme: PendexTheme = parsePendexTheme(
        {
            brand: {},
            ansi: {},
        },
        'fallback',
    );

    expect(theme.brand).toBeUndefined();
    expect(theme.ansi).toBeUndefined();
});

test('non-string entries inside a table are dropped, not thrown', (): void => {
    const theme: PendexTheme = parsePendexTheme(
        {
            brand: {
                compile: '#D6A448',
                nested: { oops: true },
                count: 5,
                flag: true,
            },
        },
        'x',
    );

    expect(theme.brand?.compile).toBe('#D6A448');
    expect(theme.brand).not.toHaveProperty('nested');
    expect(theme.brand).not.toHaveProperty('count');
    expect(theme.brand).not.toHaveProperty('flag');
});

test('non-object scalar tables degrade to undefined, while array input is treated as an indexable object by the current parser', (): void => {
    const theme: PendexTheme = parsePendexTheme(
        {
            ansi: 'not-a-table',
            git: 42,
            syntax: ['array'],
        },
        'x',
    );

    expect(theme.ansi).toBeUndefined();
    expect(theme.git).toBeUndefined();

    // Arrays are objects in JS, and the current extractTable() keeps
    // string-valued entries it can enumerate, so ['array'] becomes
    // { "0": "array" } rather than undefined.
    expect(theme.syntax).toEqual({
        0: 'array',
    });
});

```



```

});

test('normalizes valid hex values to uppercase and trims whitespace', (): void => {
    const theme: PendexTheme = parsePendexTheme(
        {
            primary: ' #aa11cc ',
            secondary: '#bb22dd',
        },
        'norm',
    );

    expect(theme.colors.primary).toBe('#AA11CC');
    expect(theme.colors.secondary).toBe('#BB22DD');
});

test('invalid hex values fall back slot-by-slot to BRAND_PALETTE', (): void => {
    const theme: PendexTheme = parsePendexTheme(
        {
            primary: '#12345', // invalid length
            secondary: 'blue', // invalid format
            success: '#12GG12', // invalid chars
            warning: 42,
        },
        'fallback-test',
    );

    expect(theme.colors.primary).toBe(BRAND_PALETTE.primary);
    expect(theme.colors.secondary).toBe(BRAND_PALETTE.secondary);
    expect(theme.colors.success).toBe(BRAND_PALETTE.success);
    expect(theme.colors.warning).toBe(BRAND_PALETTE.warning);
});

test('malformed top-level input (null, number, string, array, boolean) degrades to full brand palette, never throws', (): void => {
    const BAD_VALUES = [null, undefined, 42, 'a string', [], true] as const;

    for (const bad of BAD_VALUES) {
        const theme: PendexTheme = parsePendexTheme(bad, 'fallback-name');

        expect(theme.name).toBe('fallback-name');
        expect(theme.colors).toEqual(BRAND_PALETTE);
        expect(theme.brand).toBeUndefined();
    }
});
});

```

```

// ██████████< END >-██████████
// FILE NAME: : themepalette.test.ts
// TYPE      : .TS
// PATH      : packages/theme/tests/themepalette.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : view.test.ts
// TYPE      : .TS
// PATH      : packages/compile/tests/view.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< START >-██████████
// FILE-PATH: packages/compile/tests/view.test.ts
//
/**
 * @file packages/compile/tests/view.test.ts
 * @description Unit tests for `@pendex/compile` CompileView rendering pipeline, task building,

```

```

* category formatting, and error boundary handling. Scoped strictly to packages/compile/tests/.
*/

import type { Config, State, View } from '@pendex/core';
import { resolveRunnerDeps } from '@pendex/core';
import { FallbackTheme } from '@pendex/theme';
import {
  afterAll,
  beforeAll,
  beforeEach,
  describe,
  expect,
  spyOn,
  test,
} from 'bun:test';
import { mkdirSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';
import { CompileView } from '../src';
import * as CompileServiceModule from '../src/CompileService';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
  PACKAGE_TESTS_DIR,
  `test-sandbox-filescanner-${process.pid}`,
);

const ORIGINAL_CWD: string = process.cwd();

let state: State;
let baseConfig: () => Config;

describe('CompileView', (): void => {
  beforeAll(async (): Promise<void> => {
    state = await resolveRunnerDeps();
    baseConfig = (): Config => ({
      theme: state.config.theme,
      outputDir: 'OUT',
      rebuiltDir: 'REBUILT',
      exclude: [],
      jobs: [
        {
          filename: '1_SOURCE_FILES.txt',
          category: 'source',
          description: 'Source files',
          include: ['src/**/*.ts'],
          exclude: [],
        },
        {
          filename: '8_MISC_FILES.txt',
          category: 'misc',
          description: '',
          include: [],
          exclude: [],
        },
      ],
    });

    // Create sandbox ONCE per file suite
    await createSandbox(SANDBOX_DIR);
    process.chdir(SANDBOX_DIR);
  });

  afterAll(async (): Promise<void> => {

```

```

    // Tear down sandbox ONCE after all tests in this file complete
    process.chdir(ORIGINAL_CWD);
    await cleanupSandbox(SANDBOX_DIR);
  });

  beforeEach(() : void => {
    mkdirSync('src', { recursive: true });
    writeFileSync('src/index.ts', 'console.log("hello");\n');
  });

  // =====
  // *Error Boundary Tests
  // =====

  test('catches errors instead of throwing (invalid outputDir)', async () : Promise<void> => {
    const config: Config = baseConfig();
    config.outputDir = 'OUT\u0000BAD';

    const view = new CompileView({
      theme: FallbackTheme,
      config,
    });

    expect(view.render()).resolves.toBeUndefined();
  });

  test('catches non-Error exceptions in catch block', async () : Promise<void> => {
    const config: Config = baseConfig();
    const initSpy = spyOn(
      CompileServiceModule,
      'initializeCompile',
    ).mockImplementation(() => {
      throw 'String error thrown';
    });

    const view = new CompileView({
      theme: FallbackTheme,
      config,
    });

    expect(view.render()).resolves.toBeUndefined();
    initSpy.mockRestore();
  });

  // =====
  // * Happy Path Rendering & Task Execution
  // =====

  test('renders a complete compile session with jobs and task execution', async () : Promise<void> => {
    const config: Config = baseConfig();

    const view: View = new CompileView({
      theme: FallbackTheme,
      config,
    });

    expect(view.render()).resolves.toBeUndefined();
    expect(await Bun.file('OUT/manifest.json').exists()).toBe(true);
  });

  // =====
  // * Direct Task Building & Callback Execution
  // =====

```

```

test('builds and executes job tasks directly, updating totalFiles and formatters', async (): Promise<void> => {
    const config: Config = baseConfig();
    const view = new CompileView({
        theme: FallbackTheme,
        config,
    });

    const ctx = await CompileServiceModule.initializeCompile(config);

    const viewInternal = view as unknown as {
        buildJobTasks: (
            ctx: CompileServiceModule.CompileJobContext,
        ) => Promise<Array<{ title: string; task: () => Promise<string> }>>;
        renderSummary: (fileCount: number) => void;
        directoryFormatter: (text: string) => string;
        numberFormatter: (num: number) => string;
        mutedFormatter: (text: string) => string;
        successFormatter: (text: string) => string;
    };

    const tasksList = await viewInternal.buildJobTasks(ctx);

    expect(tasksList).toBeArray();
    expect(tasksList.length).toBe(2);

    expect(tasksList[0]!.title).toContain('Source files');
    const task1Result = await tasksList[0]!.task();
    expect(typeof task1Result).toBe('string');

    expect(tasksList[1]!.title).toContain('8_MISC_FILES.txt');
    const task2Result = await tasksList[1]!.task();
    expect(typeof task2Result).toBe('string');

    viewInternal.renderSummary(10);
    expect(viewInternal.directoryFormatter('OUT')).toContain('OUT');
    expect(viewInternal.numberFormatter(42)).toContain('42');
    expect(viewInternal.mutedFormatter('test')).toBeDefined();
    expect(viewInternal.successFormatter('test')).toBeDefined();
});

});

// =====< END >-=====
// FILE NAME: : view.test.ts
// TYPE      : .TS
// PATH      : packages/compile/tests/view.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : compile.test.ts
// TYPE      : .TS
// PATH      : packages/compile/tests/compile.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/compile/tests/compile.test.ts
//
/**
 * @file packages/compile/tests/compile.test.ts
 * @description Unit test suite for `Compile.ts`. Scoped strictly to packages/compile/tests/.
 */

import { resolveRunnerDeps, type State } from '@pendex/core';

```

```

import { afterAll, beforeAll, describe, expect, spyOn, test } from 'bun:test';
import { join } from 'node:path';
import { Compile } from '../src/Compile';
import { CompileView } from '../src/CompileView';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
  PACKAGE_TESTS_DIR,
  `test-sandbox-filescanner-${process.pid}`,
);

const ORIGINAL_CWD: string = process.cwd();
let state: State;

describe('Compile Command (Compile.ts)', (): void => {
  beforeAll(async (): Promise<void> => {
    state = await resolveRunnerDeps();
    // Create sandbox ONCE per file suite
    await createSandbox(SANDBOX_DIR);
    process.chdir(SANDBOX_DIR);
  });

  afterAll(async (): Promise<void> => {
    // Tear down sandbox ONCE after all tests in this file complete
    process.chdir(ORIGINAL_CWD);
    await cleanupSandbox(SANDBOX_DIR);
  });

  // =====
  // *Instantiation & Metadata Properties
  // =====

  test('should correctly instantiate and set key, label, and hint properties', (): void => {
    const compileCommand = new Compile(state);

    expect(compileCommand.key).toBe('compile');
    expect(compileCommand.label).toBeDefined();
    expect(compileCommand.hint).toBeDefined();

    expect(compileCommand.label).toContain('Compile project');
    expect(compileCommand.hint).toContain('Consolidates project files');
  });

  // =====
  // *Execute Method & View Delegation
  // =====

  test('should delegate execution directly to CompileView.render()', async (): Promise<void> => {
    const compileCommand = new Compile(state);

    const viewInstance = (
      compileCommand as unknown as { view: CompileView }
    ).view;
    const renderSpy = spyOn(viewInstance, 'render').mockResolvedValue(
      undefined,
    );

    await compileCommand.execute();

    expect(renderSpy).toHaveBeenCalledTimes(1);
  });
});

```

```
//                                     < END >-
// FILE NAME: : compile.test.ts
// TYPE      : .TS
// PATH      : packages/compile/tests/compile.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// _____< END >-_____

// _____< START >-_____
// FILE NAME: : compileservice.test.ts
// TYPE      : .TS
// PATH      : packages/compile/tests/compileservice.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// _____< START >-_____
// FILE-PATH: packages/compile/tests/compileservice.test.ts
//
// Proves @pendex/compile is independently testable —no ConfigManager,
// no @pendex/split, just a Config literal and this package's own
// exports. The full compile-then-split round trip lives in the root
// pendex package's integration tests, where both packages are wired
// together; this file only needs @pendex/compile to be true.
//
/**
 * @file packages/compile/tests/compileservice.test.ts
 * @description Comprehensive unit & integration tests for `@pendex/compile` CompileService,
 * achieving 100% test coverage across job execution, lifecycle hooks, stateful path claims,
 * and helper utilities.
 */

import type { Config, Manifest, State } from '@pendex/core';
import { Constants, resolveRunnerDeps } from '@pendex/core';
import {
    afterAll,
    beforeAll,
    beforeEach,
    describe,
    expect,
    mock,
    test,
    type Mock,
} from 'bun:test';
import { existsSync, mkdirSync, rmSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';
import {
    compileJob,
    compileSingleJob,
    finalizeCompile,
    initializeCompile,
    prepareOutputDirectory,
    resolveExcludes,
    runCompile,
    type CompileJob,
    type CompileJobContext,
    type CompileJobResult,
} from '../src';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
    PACKAGE_TESTS_DIR,
    `test-sandbox-filescanner-${process.pid}`,
);

const ORIGINAL_CWD: string = process.cwd();
let state: State;
```

```

let baseConfig: () => Config;

describe('@pendex/compile —CompileService (standalone)', (): void => {
  beforeAll(async (): Promise<void> => {
    state = await resolveRunnerDeps();

    baseConfig = (): Config => ({
      theme: state.config.theme,
      outputDir: 'OUT',
      rebuiltDir: 'REBUILT',
      exclude: [
        'OUT/**',
        'REBUILT/**',
        '**/node_modules/**',
        '**/ALL/**',
        '**/ALL_REBUILT/**',
        '**/.vscode/**',
        '**/coverage/**',
        '**/logs/**',
      ],
      jobs: [
        {
          filename: '1_SOURCE_FILES.txt',
          category: 'source',
          description: 'Source files',
          include: ['**/src/**/*.ts', '**/*.ts'],
          exclude: ['**/tests/**', '**/*.test.ts', '**/*.spec.ts'],
        },
        {
          filename: '8_MISC_FILES.txt',
          category: 'misc',
          description: 'Remainder files',
          include: [],
          exclude: ['**/node_modules/**', '**/tests/**'],
        },
      ],
    });

    // Create sandbox ONCE per file suite
    await createSandbox(SANDBOX_DIR);
    process.chdir(SANDBOX_DIR);
  });

  afterAll(async (): Promise<void> => {
    // Tear down sandbox ONCE after all tests in this file complete
    process.chdir(ORIGINAL_CWD);
    await cleanupSandbox(SANDBOX_DIR);
  });

  beforeEach(): void => {
    // Reset output directory and test fixtures inside the existing sandbox
    if (existsSync('OUT')) {
      rmSync('OUT', { recursive: true, force: true });
    }

    mkdirSync('src', { recursive: true });
    writeFileSync('src/index.ts', 'console.log("hello");\n');
    writeFileSync('readme.md', '# readme\n');
    writeFileSync(Constants.GITIGNORE_PATH, 'dist/\ncoverage/\n');
  });

  // =====
  // *Core Compilation & Output Resolution

```

```
// =====

test('resolveExcludes merges .gitignore with config excludes', async (): Promise<void> => {
  const excludes: string[] = await resolveExcludes(baseConfig());

  expect(excludes).toContain('OUT/**');
  expect(excludes).toBeArray();
  expect(excludes.length).toBeGreaterThan(baseConfig().exclude.length);
});

test('prepareOutputDirectory wipes and recreates the output path', async (): Promise<void> => {
  mkdirSync('OUT/sub', { recursive: true });
  writeFileSync('OUT/sub/file.txt', 'junk');

  await prepareOutputDirectory('OUT');

  expect(existsSync('OUT')).toBe(true);
  expect(existsSync('OUT/sub/file.txt')).toBe(false);
});

test('runCompile writes archives, manifest, and per-job categories', async (): Promise<void> => {
  await runCompile(baseConfig());

  const absoluteManifestPath: string = join(
    process.cwd(),
    'OUT',
    'manifest.json',
  );
  const manifest: Manifest = await Bun.file(absoluteManifestPath).json();

  expect(manifest.files['1_SOURCE_FILES.txt']).toBeDefined();
  expect(manifest.files['1_SOURCE_FILES.txt']).toBeArray();
  expect(String(manifest.files['1_SOURCE_FILES.txt'])).toContain(
    'src/index.ts',
  );
  expect(manifest.categories?.['1_SOURCE_FILES.txt']).toBe('source');
  expect(manifest.categories?.['8_MISC_FILES.txt']).toBe('misc');
});

test('compileJob returns fileCount 0 for a job matching nothing', async (): Promise<void> => {
  mkdirSync('OUT', { recursive: true });
  const manifest: Manifest = {
    files: {},
    emptyDirectories: [],
  };

  const testJob: CompileJob = {
    job: {
      filename: 'none.txt',
      category: 'web',
      description: 'empty job',
      include: ['**/*.nope'],
      exclude: [],
    },
    excludes: [],
    outputDir: 'OUT',
    manifest,
    claimedPaths: new Set<string>(),
    config: baseConfig(),
    hooks: {},
    cwd: ORIGINAL_CWD,
  };
});
```



```

    const outcome: CompileJobResult = await compileJob(testJob);

    expect(outcome.fileCount).toBe(0);
  });

  // =====
  //  Lifecycle Hooks Execution
  // =====

  test('triggers all lifecycle hooks during compile workflow', async (): Promise<void> => {
    const onCompileStartMock: Mock<() => void> = mock();
    const onJobStartMock: Mock<() => void> = mock();
    const onJobSuccessMock: Mock<() => void> = mock();
    const onCompileSuccessMock: Mock<() => void> = mock();

    const config: Config = baseConfig();

    await runCompile(config, {
      onCompileStart: onCompileStartMock,
      onJobStart: onJobStartMock,
      onJobSuccess: onJobSuccessMock,
      onCompileSuccess: onCompileSuccessMock,
    });

    expect(onCompileStartMock).toHaveBeenCalled();
    expect(onJobStartMock).toHaveBeenCalled();
    expect(onJobSuccessMock).toHaveBeenCalled();
    expect(onCompileSuccessMock).toHaveBeenCalled();
  });

  // =====
  //  Stateful Registry & Remainder Protection
  // =====

  test('claimedPaths set prevents duplicate file inclusion across jobs', async (): Promise<void> => {
    const claimedPaths: Set<string> = new Set<string>();
    const manifest: Manifest = {
      files: {},
      emptyDirectories: [],
    };

    const job1: CompileJob = {
      job: {
        filename: '1_SOURCE_FILES.txt',
        category: 'source',
        description: 'Source',
        include: ['**/src/**/*.ts', '**/*.ts'],
        exclude: ['**/tests/**'],
      },
      excludes: [],
      outputDir: 'OUT',
      manifest,
      claimedPaths,
      config: baseConfig(),
      hooks: {},
      cwd: process.cwd(),
    };

    const result1: CompileJobResult = await compileJob(job1);
    expect(result1.fileCount).toBeGreaterThan(0);

    const job2: CompileJob = {
      job: {

```

```

        filename: '2_SOURCE_FILES.txt',
        category: 'source',
        description: 'Source Overlap',
        include: ['**/src/**/*.ts', '**/*.ts'],
        exclude: [],
    },
    excludes: Array.from(claimedPaths),
    outputDir: 'OUT',
    manifest,
    claimedPaths,
    config: baseConfig(),
    hooks: {},
    cwd: process.cwd(),
};

const result2: CompileJobResult = await compileJob(job2);
expect(result2.fileCount).toBe(0);
});

// =====
//  Stepwise Compilation APIs
// =====

test('supports stepwise manual compilation via initializeCompile, compileSingleJob, and finalizeCompile', async (): Promise<void> => {
    mkdirSync('OUT', { recursive: true });
    const config: Config = baseConfig();

    const ctx: CompileJobContext = await initializeCompile(
        config,
        {},
        process.cwd(),
    );
    expect(ctx.excludes).toBeArray();
    expect(ctx.manifest).toBeDefined();

    const job = config.jobs[0!];
    const outcome: CompileJobResult = await compileSingleJob(job, ctx);
    expect(outcome.fileCount).toBeGreaterThan(0);

    const summary = await finalizeCompile(config, ctx);
    expect(summary.excludes).toEqual(ctx.excludes);
    expect(existsSync('OUT/manifest.json')).toBe(true);
});

// =====
//  Empty Directories & Directory Persistence
// =====

test('captures empty directories into manifest when present', async (): Promise<void> => {
    mkdirSync('src/empty-subfolder', { recursive: true });

    await runCompile(baseConfig());

    const manifestPath: string = join(
        process.cwd(),
        'OUT',
        'manifest.json',
    );
    const manifest: Manifest = await Bun.file(manifestPath).json();

    expect(manifest.emptyDirectories).toBeArray();
    const posixDirs = manifest.emptyDirectories.map((d: string) =>
        d.replace(/\\/g, '/'),
    );

```

```

    );
    expect(
      posixDirs.some((d: string) => d.endsWith('src/empty-subfolder')),
    ).toBe(true);
  });

  test('creates output directory recursively if it does not exist', async (): Promise<void> => {
    const config: Config = baseConfig();
    config.outputDir = 'CUSTOM_NESTED_OUT/DEPTH_2';

    await runCompile(config);

    expect(existsSync('CUSTOM_NESTED_OUT/DEPTH_2/manifest.json')).toBe(
      true,
    );
  });
});

```

```

// =====< END >-=====
// FILE NAME: : compileservice.test.ts
// TYPE      : .TS
// PATH      : packages/compile/tests/compileservice.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

```

```

// =====< START >-=====
// FILE NAME: : view.test.ts
// TYPE      : .TS
// PATH      : packages/exit/tests/view.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/exit/tests/view.test.ts
//

```

```

/**
 * @file packages/exit/tests/view.test.ts
 * @description Unit tests for `ExitView.ts` and the base `View`
 * rendering pipeline, category styling,
 * and formatter options.
 */

```

```

import {
  resolveRunnerDeps,
  View,
  type Category,
  type ExitState,
  type State,
  type StyleFunction,
} from '@pendex/core';
import { beforeAll, describe, expect, test } from 'bun:test';
import { ExitView } from '../src/ExitView';

```

```

let state: State;
let exitState: ExitState;

```

```

beforeAll(async (): Promise<void> => {
  state = await resolveRunnerDeps();
  exitState = {
    theme: state.theme,
    config: state.config,
    exit: () => {
      throw new Error('Exit called');
    },
  },

```

```

        exitCode: 0,
        categoryColors: {
            source: '#799470',
        },
    };
});

describe('ExitView', (): void => {
    test('should instantiate and render ExitView directly', async (): Promise<void> => {
        const exitView = new ExitView(exitState);
        await exitView.render();
        expect(exitView).toBeInstanceOf(ExitView);
    });

    test('should execute base View.prototype.render method', async (): Promise<void> => {
        const exitView = new ExitView(exitState);

        // Explicitly invokes base View.prototype.render
        await View.prototype.render.call(exitView);

        const baseView = new View(state);
        await baseView.render();
        expect(baseView).toBeInstanceOf(View);
    });

    test('should style text using categoryStyle with category hex and fallback', (): void => {
        const exitView = new ExitView(exitState);

        const categoryStyleFn = (
            exitView as unknown as {
                categoryStyle: (
                    category: Category,
                    fallback: StyleFunction,
                ) => StyleFunction;
            }
        ).categoryStyle.bind(exitView);

        const fallbackFn: StyleFunction = (text: string): string =>
            `FALLBACK:${text}`;

        // 1. Branch: hex is present in categoryColors ('source' -> '#799470')
        const sourceStyler: StyleFunction = categoryStyleFn(
            'source',
            fallbackFn,
        );
        const sourceResult: string = sourceStyler('SourceCode');

        expect(sourceResult).toContain('SourceCode');
        expect(sourceResult).not.toContain('FALLBACK:');

        // 2. Branch: hex is absent in categoryColors ('web' has no hex in categoryColors)
        const webStyler: StyleFunction = categoryStyleFn('web', fallbackFn);
        const webResult: string = webStyler('WebCode');

        expect(webResult).toBe('FALLBACK:WebCode');
    });
});

// =====< END >=====
// FILE NAME : view.test.ts
// TYPE      : .TS
// PATH      : packages/exit/tests/view.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z

```

```

//===== < END >=====

//===== < START >=====
// FILE NAME: : exit.test.ts
// TYPE      : .TS
// PATH      : packages/exit/tests/exit.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
//===== < START >=====
// FILE-PATH: packages/exit/tests/exit.test.ts
//

/**
 * @file packages/exit/tests/exit.test.ts
 * @description Unit test suite for `Exit.ts`, achieving 100% test &
branch coverage across
 * command property initialization, custom exit callbacks, and default
process exit fallbacks.
 */

import { resolveRunnerDeps, type ExitState, type State } from '@pendex/core';
import {
  beforeAll,
  beforeEach,
  describe,
  expect,
  mock,
  spyOn,
  test,
  type Mock,
} from 'bun:test';
import { Exit } from '../src/Exit';

type ExitFn = ExitState['exit'];
type ExitCode = number | string | null | undefined;

let exitMock: Mock<ExitFn>;
let state: State;
let exitState: ExitState;
let exitCmd: Exit;

beforeAll(async (): Promise<void> => {
  state = await resolveRunnerDeps();
});

beforeEach(): void => {
  exitMock = mock((_code?: ExitCode): never => {
    return undefined as never;
  });
};

exitState = {
  theme: state.theme,
  config: {
    theme: state.config.theme,
    outputDir: 'ALL',
    rebuiltDir: 'ALL_REBUILT',
    exclude: [],
    jobs: [],
  },
  exit: exitMock,
  exitCode: 0,
};

exitCmd = new Exit(exitState);

```

```

});

describe('Exit Command (Exit.ts)', (): void => {
  test('should have correct key, label, and hint properties', (): void => {
    expect(exitCmd.key).toBe('exit');
    expect(exitCmd.label).toBeDefined();
    expect(exitCmd.hint).toBeDefined();

    expect(exitCmd.label).toContain('Exit program');
    expect(exitCmd.hint).toContain('Terminate system process');
  });

  test('should call custom exit function with code 0 when exit state property is provided', async (): Promise<void> => {
    await exitCmd.execute();

    expect(exitMock).toHaveBeenCalledWith(0);
  });

  test('should fallback to process.exit when state.exit function is not provided', async (): Promise<void> => {
    const processExitSpy = spyOn(process, 'exit').mockImplementation(
      (_code?: number): never => {
        return undefined as never;
      },
    );

    const fallbackState = {
      theme: state.theme,
      config: exitState.config,
      exitCode: 0,
    } as ExitState;

    delete (fallbackState as { exit?: ExitFn }).exit;

    const fallbackExitCmd = new Exit(fallbackState);
    await fallbackExitCmd.execute();

    expect(processExitSpy).toHaveBeenCalledWith(0);
    processExitSpy.mockRestore();
  });
});

// =====< END >-=====
// FILE NAME: : exit.test.ts
// TYPE      : .TS
// PATH      : packages/exit/tests/exit.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : colors.test.ts
// TYPE      : .TS
// PATH      : packages/color/tests/colors.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/color/tests/color.test.ts
//
// oxlint-disable typescript/no-explicit-any
//
/**
 * @file packages/color/tests/color.test.ts
 * @description Unit test suite for internal color utilities, detection logic, SGR modifiers,
 * truecolor ANSI parsing, and runtime robustness across the `@pendex/color` package.
 */

```

```

import { afterEach, describe, expect, test } from 'bun:test';

import {
  bg,
  Colors,
  detectColorSupport,
  ESC,
  fg,
  parseHex,
  style,
  type StyleFactory,
  type Styler,
} from '../src';

describe('Colors Class', (): void => {
  // Save original state of environment variables & stdout
  const originalEnv = { ...Bun.env };
  const originalStdoutTTY: boolean | undefined = process.stdout?.isTTY;

  afterEach((): void => {
    // Restore environment variables and settings after each test
    for (const key in Bun.env) {
      delete Bun.env[key];
    }

    Object.assign(Bun.env, originalEnv);

    if (process.stdout) {
      Object.defineProperty(process.stdout, 'isTTY', {
        value: originalStdoutTTY,
        configurable: true,
      });
    }

    Colors.enable(); // Default back to enabled
  });

  // =====
  // *Environment Detection & State Controls
  // =====
  test('should successfully test internal color functions', (): void => {
    const enabled: boolean = detectColorSupport();
    expect(enabled).toBe(true);

    const myStyle: Styler = style(`${ESC}31m`, `${ESC}39m`);
    const myStyleResult: string = myStyle('Test');
    expect(myStyleResult).toBe(`${ESC}31mTest${ESC}39m`);

    const newStyle: StyleFactory = (
      open: string,
      close: string,
    ): Styler => {
      return (text: string): string => {
        if (!enabled) return text;

        // Handle cases where runtime data does not match TypeScript types
        if (typeof text !== 'string') {
          return `${open}${String(text ?? '').replaceAll(close, open)}${close}`;
        }

        return `${open}${text.replaceAll(close, open)}${close}`;
      };
    };
  });

```

```

    };
    const starStyle: Styler = newStyle('*', '*');
    const starResult: string = starStyle('STARS');
    expect(starResult).toBe('*STARS*');

    const redFgStyler: Styler = fg(31);
    expect(redFgStyler('Test')).toBe(`${ESC}31mTest${ESC}39m`);

    const redBgStyler: Styler = bg(41);
    expect(redBgStyler('Test')).toBe(`${ESC}41mTest${ESC}49m`);

    const [r, g, b] = parseHex('#ff8800');
    expect(r).toBe(255);
    expect(g).toBe(136);
    expect(b).toBe(0);

    const customOrange: Styler = Colors.hex('#ff8800');
    expect(customOrange('Orange')).toBe(
        `${ESC}38;2;255;136;0mOrange${ESC}39m`,
    );
});

test('should honor NO_COLOR convention', (): void => {
    delete Bun.env.FORCE_COLOR;
    Bun.env.NO_COLOR = '1';

    Colors.disable();
    expect(Colors.isEnabled).toBe(false);
});

test('should honor FORCE_COLOR when set to non-zero values', (): void => {
    Bun.env.FORCE_COLOR = '1';
    Colors.enable();
    expect(Colors.isEnabled).toBe(true);
});

test('should respect manual enablement and disablement', (): void => {
    Colors.disable();
    expect(Colors.isEnabled).toBe(false);
    expect(Colors.red('text')).toBe('text');

    Colors.enable();
    expect(Colors.isEnabled).toBe(true);
    expect(Colors.red('text')).toBe('\x1b[31mtext\x1b[39m');
});

// =====
//   Standard Stylers
// =====

test('should apply SGR modifiers correctly', (): void => {
    Colors.enable();
    expect(Colors.reset('text')).toBe('\x1b[0mtext\x1b[0m');
    expect(Colors.bold('text')).toBe('\x1b[1mtext\x1b[22m');
    expect(Colors.dim('text')).toBe('\x1b[2mtext\x1b[22m');
    expect(Colors.italic('text')).toBe('\x1b[3mtext\x1b[23m');
    expect(Colors.underline('text')).toBe('\x1b[4mtext\x1b[24m');
    expect(Colors.inverse('text')).toBe('\x1b[7mtext\x1b[27m');
    expect(Colors.strikethrough('text')).toBe('\x1b[9mtext\x1b[29m');
});

test('should apply foreground colors correctly', (): void => {
    Colors.enable();

```



```

    expect(Colors.black('text')).toBe('\x1b[30mtext\x1b[39m');
    expect(Colors.red('text')).toBe('\x1b[31mtext\x1b[39m');
    expect(Colors.green('text')).toBe('\x1b[32mtext\x1b[39m');
    expect(Colors.yellow('text')).toBe('\x1b[33mtext\x1b[39m');
    expect(Colors.blue('text')).toBe('\x1b[34mtext\x1b[39m');
    expect(Colors.magenta('text')).toBe('\x1b[35mtext\x1b[39m');
    expect(Colors.cyan('text')).toBe('\x1b[36mtext\x1b[39m');
    expect(Colors.white('text')).toBe('\x1b[37mtext\x1b[39m');
  });

  test('should apply bright foreground colors correctly', (): void => {
    Colors.enable();
    expect(Colors.gray('text')).toBe('\x1b[90mtext\x1b[39m');
    expect(Colors.blackBright('text')).toBe('\x1b[90mtext\x1b[39m');
    expect(Colors.redBright('text')).toBe('\x1b[91mtext\x1b[39m');
    expect(Colors.greenBright('text')).toBe('\x1b[92mtext\x1b[39m');
    expect(Colors.yellowBright('text')).toBe('\x1b[93mtext\x1b[39m');
    expect(Colors.blueBright('text')).toBe('\x1b[94mtext\x1b[39m');
    expect(Colors.magentaBright('text')).toBe('\x1b[95mtext\x1b[39m');
    expect(Colors.cyanBright('text')).toBe('\x1b[96mtext\x1b[39m');
    expect(Colors.whiteBright('text')).toBe('\x1b[97mtext\x1b[39m');
  });

  test('should apply background colors correctly', (): void => {
    Colors.enable();
    expect(Colors.bgBlack('text')).toBe('\x1b[40mtext\x1b[49m');
    expect(Colors.bgRed('text')).toBe('\x1b[41mtext\x1b[49m');
    expect(Colors.bgGreen('text')).toBe('\x1b[42mtext\x1b[49m');
    expect(Colors.bgYellow('text')).toBe('\x1b[43mtext\x1b[49m');
    expect(Colors.bgBlue('text')).toBe('\x1b[44mtext\x1b[49m');
    expect(Colors.bgMagenta('text')).toBe('\x1b[45mtext\x1b[49m');
    expect(Colors.bgCyan('text')).toBe('\x1b[46mtext\x1b[49m');
    expect(Colors.bgWhite('text')).toBe('\x1b[47mtext\x1b[49m');
  });

  test('should apply bright background colors correctly', (): void => {
    Colors.enable();
    expect(Colors.bgBlackBright('text')).toBe('\x1b[100mtext\x1b[49m');
    expect(Colors.bgRedBright('text')).toBe('\x1b[101mtext\x1b[49m');
    expect(Colors.bgGreenBright('text')).toBe('\x1b[102mtext\x1b[49m');
    expect(Colors.bgYellowBright('text')).toBe('\x1b[103mtext\x1b[49m');
    expect(Colors.bgBlueBright('text')).toBe('\x1b[104mtext\x1b[49m');
    expect(Colors.bgMagentaBright('text')).toBe('\x1b[105mtext\x1b[49m');
    expect(Colors.bgCyanBright('text')).toBe('\x1b[106mtext\x1b[49m');
    expect(Colors.bgWhiteBright('text')).toBe('\x1b[107mtext\x1b[49m');
  });

  // =====
  //  Truecolor (24-bit Hex)
  //  =====

  test('should parse and apply 6-digit hex colors with and without hash', (): void => {
    Colors.enable();
    const fgHexWithHash: Styler = Colors.hex('#ff8800');
    const fgHexWithoutHash: Styler = Colors.hex('ff8800');

    expect(fgHexWithHash('truecolor')).toBe(
      '\x1b[38;2;255;136;0mtruecolor\x1b[39m',
    );
    expect(fgHexWithoutHash('truecolor')).toBe(
      '\x1b[38;2;255;136;0mtruecolor\x1b[39m',
    );
  });

```

```

    const bgHexWithHash: Styler = Colors.bgHex('#00ff88');
    expect(bgHexWithHash('truecolor')).toBe(
      '\x1b[48;2;0;255;136mtruecolor\x1b[49m',
    );
  });

  test('should parse and expand 3-digit shorthand hex colors', (): void => {
    Colors.enable();
    const fgShorthand: Styler = Colors.hex('#f80');
    expect(fgShorthand('shorthand')).toBe(
      '\x1b[38;2;255;136;0mshorthand\x1b[39m',
    );
  });

  test('should throw an error for invalid hex color values', (): void => {
    expect(() : void => {
      Colors.hex('#invalid');
    }).toThrow('Colors: invalid hex color "#invalid"');
    expect(() : void => {
      Colors.hex('1234567');
    }).toThrow('Colors: invalid hex color "1234567"');
    expect(() : void => {
      Colors.hex('12');
    }).toThrow('Colors: invalid hex color "12"');
  });

  test('should instantiate a new Colors object', (): void => {
    // @ts-expect-error
    const MyColors: Colors = new (Colors as unknown)();

    expect(MyColors).toBeInstanceOf(Colors);
  });
});

```

```

// ██████████< END >────────────────────────────────────────────────────────────────────────────────
// FILE NAME: : colors.test.ts
// TYPE      : .TS
// PATH      : packages/color/tests/colors.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< END >────────────────────────────────────────────────────────────────────────────────

```

```

// ██████████< START >────────────────────────────────────────────────────────────────────────────────
// FILE NAME: : view.test.ts
// TYPE      : .TS
// PATH      : packages/split/tests/view.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< START >────────────────────────────────────────────────────────────────────────────────
// FILE-PATH: packages/split/tests/view.test.ts
//

```

```

/**
 * @file packages/split/tests/view.test.ts
 * @description Unit tests for `@pendex/split` SplitView rendering
 * pipeline, task building,
 * category formatting, and error boundary handling. Scoped strictly
 * to packages/split/tests/.
 */

```

```

import type { Config, Manifest, State } from '@pendex/core';
import {
  buildArchiveEntry,
  joinArchiveEntries,
  resolveRunnerDeps,

```

```

} from '@pendex/core';
import { FallbackTheme } from '@pendex/theme';
import {
  afterAll,
  beforeAll,
  beforeEach,
  describe,
  expect,
  spyOn,
  test,
} from 'bun:test';
import { mkdirSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';
import { SplitView } from '../src';
import * as SplitServiceModule from '../src/SplitService';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
  PACKAGE_TESTS_DIR,
  `test-sandbox-filescanner-${process.pid}`,
);

const ORIGINAL_CWD: string = process.cwd();

let state: State;
let baseConfig: () => Config;

beforeAll(async (): Promise<void> => {
  state = await resolveRunnerDeps();
  baseConfig = (): Config => ({
    theme: state.config.theme,
    outputDir: 'OUT',
    rebuiltDir: 'REBUILT',
    exclude: ['OUT/**', 'REBUILT/**'],
    jobs: [],
  });

  await createSandbox(SANDBOX_DIR);
  process.chdir(SANDBOX_DIR);
});

afterAll(async (): Promise<void> => {
  process.chdir(ORIGINAL_CWD);
  await cleanupSandbox(SANDBOX_DIR);
});

describe('SplitView', (): void => {
  beforeEach(): void => {
    mkdirSync('OUT', { recursive: true });

    const archive = joinArchiveEntries([
      buildArchiveEntry('src/b.ts', 'export const b = 2;\n'),
    ]);
    writeFileSync(join('OUT', 'code.txt'), archive);
    writeFileSync(
      join('OUT', 'manifest.json'),
      JSON.stringify({
        files: {
          'code.txt': ['src/b.ts'],
          'missing.txt': ['src/missing.ts'],
        },
        categories: { 'code.txt': 'source' },
        emptyDirectories: [],
      })
    );
  }
});

```

```

    }),
  );
});

// =====
// *Error Boundaries & Manifest Checks
// =====

test('catches errors instead of throwing (invalid empty-dir name in manifest)', async (): Promise<void> => {
  writeFileSync(
    'OUT/manifest.json',
    JSON.stringify({
      files: {},
      emptyDirectories: ['bad\u0000dir'],
    }),
  );

  const view = new SplitView({
    theme: FallbackTheme,
    config: baseConfig(),
  });

  expect(view.render()).resolves.toBeUndefined();
});

test('reports missing manifest without throwing', async (): Promise<void> => {
  const config = baseConfig();
  config.outputDir = 'NON_EXISTENT';

  const view = new SplitView({
    theme: FallbackTheme,
    config,
  });

  expect(view.render()).resolves.toBeUndefined();
});

test('catches non-Error exceptions in catch block', async (): Promise<void> => {
  const initSpy = spyOn(
    SplitServiceModule,
    'initializeSplit',
  ).mockImplementation(() => {
    throw 'String error thrown';
  });

  const view = new SplitView({
    theme: FallbackTheme,
    config: baseConfig(),
  });

  expect(view.render()).resolves.toBeUndefined();
  initSpy.mockRestore();
});

// =====
// * Happy Path Rendering
// =====

test('renders a complete split session successfully', async (): Promise<void> => {
  const view = new SplitView({
    theme: FallbackTheme,
    config: baseConfig(),
  });

```

```

    expect(view.render()).resolves.toBeUndefined();
    expect(await Bun.file('REBUILT/src/b.ts').exists()).toBe(true);
  });

  // =====
  //  Direct Task Building & Callback Execution
  //  =====

  test('builds and executes file tasks directly, handling missing source archives and formatters', async (): Promise<void> => {
    const view = new SplitView({
      theme: FallbackTheme,
      config: baseConfig(),
    });

    const manifest: Manifest = {
      files: {
        'code.txt': ['src/b.ts'],
        'missing.txt': ['src/missing.ts'],
      },
      categories: { 'code.txt': 'source' },
      emptyDirectories: [],
    };

    const outcomes: number[] = [];

    const viewInternal = view as unknown as {
      buildFileTask: (
        filename: string,
        index: number,
        manifest: Manifest,
        outcomes: number[],
      ) => { title: string; task: () => Promise<unknown> };
      mutedFormatter: (text: string) => string;
    };

    // Task 1: Existing Archive ('code.txt')
    const task1 = viewInternal.buildFileTask(
      'code.txt',
      0,
      manifest,
      outcomes,
    );
    expect(task1.title).toContain('code.txt');
    const res1 = await task1.task();
    expect(res1).toBeDefined();
    expect(outcomes[0]).toBe(1);

    // Task 2: Missing Archive ('missing.txt')
    const task2 = viewInternal.buildFileTask(
      'missing.txt',
      1,
      manifest,
      outcomes,
    );
    expect(task2.title).toContain('missing.txt');
    const res2 = await task2.task();
    expect(res2).toBeDefined();
    expect(outcomes[1]).toBe(0);

    expect(viewInternal.mutedFormatter('test')).toBeDefined();
  });
});

```

```

// =====< END >=====
// FILE NAME: : view.test.ts
// TYPE      : .TS
// PATH      : packages/split/tests/view.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >=====

// =====< START >=====
// FILE NAME: : split.test.ts
// TYPE      : .TS
// PATH      : packages/split/tests/split.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >=====
// FILE-PATH: packages/split/tests/split.test.ts
//

/**
 * @file packages/split/tests/split.test.ts
 * @description Unit test suite for `Split.ts`. Scoped strictly to packages/split/tests/.
 */

import { resolveRunnerDeps, type State } from '@pendex/core';
import { afterAll, beforeAll, describe, expect, spyOn, test } from 'bun:test';
import { join } from 'node:path';
import { Split } from '../src/Split';
import { SplitView } from '../src/SplitView';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
  PACKAGE_TESTS_DIR,
  `test-sandbox-filescanner-${process.pid}`,
);

const ORIGINAL_CWD: string = process.cwd();
let state: State;

describe('Split Command (Split.ts)', (): void => {
  beforeAll(async (): Promise<void> => {
    state = await resolveRunnerDeps();
    await createSandbox(SANDBOX_DIR);
    process.chdir(SANDBOX_DIR);
  });

  afterAll(async (): Promise<void> => {
    process.chdir(ORIGINAL_CWD);
    await cleanupSandbox(SANDBOX_DIR);
  });

  // =====
  // * Instantiation & Metadata Properties
  // =====

  test('should correctly instantiate and set key, label, and hint properties', (): void => {
    const splitCommand = new Split(state);

    expect(splitCommand.key).toBe('split');
    expect(splitCommand.label).toBeDefined();
    expect(splitCommand.hint).toBeDefined();

    expect(splitCommand.label).toContain('Split Archive');
    expect(splitCommand.hint).toContain('Rebuild project files');
  });

```

```

// =====
//   Execute Method & View Delegation
// =====

test('should delegate execution directly to SplitView.render()', async (): Promise<void> => {
    const splitCommand = new Split(state);

    const viewInstance = (splitCommand as unknown as { view: SplitView })
        .view;
    const renderSpy = spyOn(viewInstance, 'render').mockResolvedValue(
        undefined,
    );

    await splitCommand.execute();

    expect(renderSpy).toHaveBeenCalledTimes(1);
});

// =====< END >-=====
// FILE NAME: : split.test.ts
// TYPE      : .TS
// PATH      : packages/split/tests/split.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : splitservice.test.ts
// TYPE      : .TS
// PATH      : packages/split/tests/splitservice.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// FILE-PATH: packages/split/test/splitservice.test.ts
//

/**
 * @file packages/split/test/splitservice.test.ts
 * @description Comprehensive unit & integration tests for
 * @pendex/split` SplitService,
 * achieving 100% test coverage across archive parsing, lifecycle
 * hooks, empty directory restoration,
 * and stepwise execution APIs.
 */

import {
    buildArchiveEntry,
    joinArchiveEntries,
    type Manifest,
} from '@pendex/core';
import {
    afterAll,
    beforeAll,
    beforeEach,
    describe,
    expect,
    mock,
    test,
    type Mock,
} from 'bun:test';
import { existsSync, mkdirSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';
import {

```

```

initializeSplit,
prepareRebuildDirectory,
readManifest,
restoreEmptyDirectories,
runSplit,
splitArchiveFile,
splitSingleFile,
type SplitFileOutcome,
type SplitSummary,
} from '../src';
import { PACKAGE_TESTS_DIR, cleanupSandbox, createSandbox } from './preload';

const SANDBOX_DIR: string = join(
  PACKAGE_TESTS_DIR,
  `test-sandbox-filescanner-${process.pid}`,
);

const ORIGINAL_CWD: string = process.cwd();

describe('@pendex/split —SplitService (standalone)', (): void => {
  beforeAll(async (): Promise<void> => {
    await createSandbox(SANDBOX_DIR);
    process.chdir(SANDBOX_DIR);
  });

  afterAll(async (): Promise<void> => {
    process.chdir(ORIGINAL_CWD);
    await cleanupSandbox(SANDBOX_DIR);
  });

  beforeEach(() : void => {
    mkdirSync('OUT', { recursive: true });

    const archive = joinArchiveEntries([
      buildArchiveEntry('src/a.ts', 'export const a = 1;\n'),
    ]);
    writeFileSync(join('OUT', 'code.txt'), archive);
    writeFileSync(
      join('OUT', 'manifest.json'),
      JSON.stringify({
        files: { 'code.txt': ['src/a.ts'] },
        categories: { 'code.txt': 'source' },
        emptyDirectories: ['src/empty-dir'],
      }),
    );
  });

  // =====
  // *Manifest & Directory Management
  // =====

  test('readManifest reads a hand-built manifest correctly', async (): Promise<void> => {
    const manifest: Manifest = (await readManifest('OUT')) as Manifest;
    expect(manifest?.files['code.txt']).toEqual(['src/a.ts']);
    expect(manifest?.categories?.['code.txt']).toBe('source');
  });

  test('readManifest returns null if manifest.json does not exist', async (): Promise<void> => {
    const manifest = await readManifest('NO_DIR');
    expect(manifest).toBeNull();
  });

  test('prepareRebuildDirectory wipes target folder', async (): Promise<void> => {

```



```

    mkdirSync('REBUILT/temp', { recursive: true });
    writeFileSync('REBUILT/temp/file.txt', 'test');

    await prepareRebuildDirectory('REBUILT');

    expect(existsSync('REBUILT')).toBe(false);
  });

  test('restoreEmptyDirectories creates specified directories recursively', async (): Promise<void> => {
    await restoreEmptyDirectories('REBUILT', ['a/b/c', 'd']);

    expect(existsSync('REBUILT/a/b/c')).toBe(true);
    expect(existsSync('REBUILT/d')).toBe(true);
  });

  // =====
  //  Archive Splitting Logic
  // =====

  test('splitArchiveFile rebuilds the original file byte-perfect', async (): Promise<void> => {
    const outcome: SplitFileOutcome = await splitArchiveFile(
      'OUT',
      'REBUILT',
      'code.txt',
    );
    expect(outcome.archiveFound).toBe(true);
    expect(outcome.filesRecreated).toBe(1);

    const rebuilt: string = await Bun.file(
      join('REBUILT', 'src', 'a.ts'),
    ).text();
    expect(rebuilt).toBe('export const a = 1;\n');
  });

  test('splitArchiveFile returns archiveFound: false for a missing archive file', async (): Promise<void> => {
    const outcome: SplitFileOutcome = await splitArchiveFile(
      'OUT',
      'REBUILT',
      'nonexistent.txt',
    );
    expect(outcome.archiveFound).toBe(false);
    expect(outcome.filesRecreated).toBe(0);
  });

  // =====
  //  Hooks & Full Pipeline (runSplit)
  // =====

  test('runSplit executes full extraction and triggers lifecycle hooks', async (): Promise<void> => {
    const onSplitStartMock: Mock<() => void> = mock();
    const onFileStartMock: Mock<() => void> = mock();
    const onFileSuccessMock: Mock<() => void> = mock();

    const summary: SplitSummary = await runSplit('OUT', 'REBUILT', {
      onSplitStart: onSplitStartMock,
      onFileStart: onFileStartMock,
      onFileSuccess: onFileSuccessMock,
    });

    expect(summary.manifestFound).toBe(true);
    expect(summary.totalFilesRecreated).toBe(1);
    expect(summary.fileOutcomes.length).toBe(1);
  });

```

```

    expect(onSplitStartMock).toHaveBeenCalled();
    expect(onFileStartMock).toHaveBeenCalled();
    expect(onFileSuccessMock).toHaveBeenCalled();
    expect(existsSync('REBUILT/src/a.ts')).toBe(true);
    expect(existsSync('REBUILT/src/empty-dir')).toBe(true);
  });

  test('runSplit reports manifestFound: false with no manifest present', async (): Promise<void> => {
    const summary: SplitSummary = await runSplit('NO_SUCH_DIR', 'REBUILT');
    expect(summary.manifestFound).toBe(false);
    expect(summary.totalFilesRecreated).toBe(0);
  });

  // =====
  //  Stepwise Split APIs
  // =====

  test('supports stepwise manual splitting via initializeSplit and splitSingleFile', async (): Promise<void> => {
    const manifest = await initializeSplit('OUT', 'REBUILT');
    expect(manifest).not.toBeNull();
    expect(manifest?.files['code.txt']).toBeDefined();

    const outcome = await splitSingleFile('OUT', 'REBUILT', 'code.txt');
    expect(outcome.filesRecreated).toBe(1);

    const nullInit = await initializeSplit('NO_SUCH_DIR', 'REBUILT');
    expect(nullInit).toBeNull();
  });
});

// =====< END >-=====
// FILE NAME: : splitservice.test.ts
// TYPE      : .TS
// PATH      : packages/split/tests/splitservice.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : app.test.ts
// TYPE      : .TS
// PATH      : packages/utills/tests/app.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// oxlint-disable typescript/no-explicit-any

import {
  beforeEach,
  describe,
  expect,
  mock,
  spyOn,
  test,
  type Mock,
} from 'bun:test';

// Mock the module BEFORE all imports.
void mock.module('@clack/prompts', () => any => ({
  select: mock(),
  confirm: mock(),
  outro: mock(),
  intro: mock(),
  isCancel: mock((val: unknown): boolean => val === Symbol.for('cancel')),
}));

```

```

// Import the now-mocked functions and the application code.
import { confirm, select } from '@clack/prompts';
import { Compile } from '@pendex/compile';
import { Exit } from '@pendex/exit';
import { Application } from '../src/components/App';

const selectMock = select as Mock<typeof select>;
const confirmMock = confirm as Mock<typeof confirm>;

describe('Application Orchestrator (App.ts)', (): void => {
  let app: Application;
  const exitProcessMock: Mock<(code?: number | string | null) => never> =
    mock((code?: number | string | null): never => {
      throw new Error('Process exited with code: ${code}`);
    });

  beforeEach(async (): Promise<void> => {
    selectMock.mockClear();
    confirmMock.mockClear();
    exitProcessMock.mockClear();
    Application.resetInstance();

    app = Application.getInstance();
    await app.init(exitProcessMock);
  });

  test('getInstance returns the same instance until reset', (): void => {
    expect(Application.getInstance()).toBe(app);
  });

  test('should execute a command and return true if user continues', async (): Promise<void> => {
    const compileCmdKey = app.commands.find(c => c instanceof Compile)!.key;
    selectMock.mockResolvedValueOnce(compileCmdKey);
    confirmMock.mockResolvedValueOnce(true);

    const compileSpy: Mock<() => Promise<void>> = spyOn(
      app.commands.find(c => c instanceof Compile)!,
      'execute',
    ).mockResolvedValue(undefined);

    const shouldContinue = await app.runSingleIteration();

    expect(compileSpy).toHaveBeenCalledTimes(1);
    expect(shouldContinue).toBe(true);
  });

  test('should execute a command and return false if user declines', async () => {
    const compileCmdKey = app.commands.find(c => c instanceof Compile)!.key;
    selectMock.mockResolvedValueOnce(compileCmdKey);
    confirmMock.mockResolvedValueOnce(false);

    const compileSpy: Mock<() => Promise<void>> = spyOn(
      app.commands.find(c => c instanceof Compile)!,
      'execute',
    ).mockResolvedValue(undefined);

    const shouldContinue: boolean = await app.runSingleIteration();

    expect(compileSpy).toHaveBeenCalledTimes(1);
    expect(shouldContinue).toBe(false);
  });
});

```

```

test('should return false if user selects the "exit" command key', async (): Promise<void> => {
    const exitCmdKey = app.commands.find(c => c instanceof Exit)!.key;
    selectMock.mockResolvedValueOnce(exitCmdKey);

    const shouldContinue: boolean = await app.runSingleIteration();

    expect(shouldContinue).toBe(false);
});

test('should return false if user cancels the main menu prompt', async (): Promise<void> => {
    selectMock.mockResolvedValueOnce(Symbol.for('cancel'));

    const shouldContinue = await app.runSingleIteration();

    expect(shouldContinue).toBe(false);
});

test('should return true (and execute nothing) for an unrecognized command key', async (): Promise<void> => {
    // Covers the `if (command)` false branch: choice is neither
    // cancel nor "exit" nor any registered command key.
    selectMock.mockResolvedValueOnce('not-a-real-command');

    const executeSpies: Mock<() => Promise<void>>[] = app.commands.map(
        cmd => spyOn(cmd, 'execute').mockResolvedValue(undefined),
    );

    const shouldContinue: boolean = await app.runSingleIteration();

    expect(shouldContinue).toBe(true);
    for (const spy of executeSpies) {
        expect(spy).not.toHaveBeenCalled();
    }
});

test('run should loop until runSingleIteration returns false and then exit', async (): Promise<void> => {
    spyOn(app, 'init').mockResolvedValue(undefined);

    const iterationSpy: Mock<() => Promise<boolean>> = spyOn(
        app,
        'runSingleIteration',
    )
        .mockResolvedValueOnce(true)
        .mockResolvedValueOnce(false);

    const exitSpy: Mock<() => Promise<void>> = spyOn(
        app.commands.find(c => c instanceof Exit)!,
        'execute',
    ).mockResolvedValue(undefined);

    await app.run(exitProcessMock);

    expect(iterationSpy).toHaveBeenCalledTimes(2);
    expect(exitSpy).toHaveBeenCalledTimes(1);
});
});

// =====< END >-=====
// FILE NAME: : app.test.ts
// TYPE      : .TS
// PATH      : packages/utils/tests/app.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

```

```

// ██████████ < START > ██████████
// FILE NAME: : integration.test.ts
// TYPE      : .TS
// PATH      : packages/utils/tests/integration.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████ < START > ██████████
// FILE-PATH: tests/integration.test.ts
//
// CROSS-PACKAGE INTEGRATION. Everything here exercises @pendex/compile
// and @pendex/split TOGETHER —the full compile →split round trip that
// neither package can test alone (each package's own suite proves it
// works in isolation; this proves they agree on the archive format and
// manifest in practice). It lives in the root pendex package because
// that's the only package that depends on both.

import { Compile } from '@pendex/compile';
import { ConfigManager, type Config, type Manifest } from '@pendex/core';
import { Split, SplitView } from '@pendex/split';
import { FallbackTheme } from '@pendex/theme';
import { afterEach, beforeEach, describe, expect, test } from 'bun:test';
import { existsSync, mkdirSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';
import {
  ABSOLUTE_SANDBOX_PATH,
  cleanupSandbox,
  createSandbox,
} from '../../../tests/preload';

describe('Compile →Split round trip (cross-package)', (): void => {
  let testConfig: Config;
  const originalCwd: string = process.cwd();

  beforeEach(async (): Promise<void> => {
    await createSandbox(ABSOLUTE_SANDBOX_PATH);
    process.chdir(ABSOLUTE_SANDBOX_PATH);

    ConfigManager.resetInstance();
    const configManager = await ConfigManager.getInstance();
    testConfig = JSON.parse(JSON.stringify(configManager.get()));

    testConfig.outputDir = 'OUT';
    testConfig.rebuiltDir = 'REBUILT';
    testConfig.jobs = [
      {
        filename: 'code.txt',
        category: 'source',
        description: 'Source files',
        include: ['src/**/*.ts'],
        exclude: ['**/ignore-me.ts'],
      },
    ];
  });

  mkdirSync('src/utils', { recursive: true });

  // Fixtures WITH trailing newlines —verbatim round-trip must
  // preserve them byte for byte.
  writeFileSync('src/index.ts', 'console.log("hello");\n');
  writeFileSync('src/utils/helper.ts', 'export const help = () => {}; \n');
  writeFileSync('src/utils/ignore-me.ts', '/* excluded\n');
  mkdirSync(join('src', 'empty-dir'), {
    recursive: true,
  });
});

```

```

afterEach(async (): Promise<void> => {
  process.chdir(originalCwd);
  await cleanupSandbox(ABSOLUTE_SANDBOX_PATH);
  ConfigManager.resetInstance();
});

test('compiles and splits files with byte-perfect fidelity', async (): Promise<void> => {
  const state = {
    theme: FallbackTheme,
    config: testConfig,
  };
  await new Compile(state).execute();

  const manifest: Manifest = await Bun.file(
    join('OUT', 'manifest.json'),
  ).json();
  expect(manifest.files['code.txt']).toContain('src/index.ts');
  expect(manifest.categories?.['code.txt']).toBe('source');

  await new Split(state).execute();

  const originalIndex: string = await Bun.file('src/index.ts').text();
  const rebuiltIndex: string = await Bun.file(
    join('src', 'index.ts'),
  ).text();
  expect(rebuiltIndex).toBe(originalIndex);

  const originalHelper: string = await Bun.file(
    'src/utils/helper.ts',
  ).text();
  const rebuiltHelper: string = await Bun.file(
    'src/utils/helper.ts',
  ).text();
  expect(rebuiltHelper).toBe(originalHelper);

  // Excluded file must not be rebuilt:
  expect(
    await Bun.file(join('REBUILT', 'src/utils/ignore-me.ts')).exists(),
  ).toBe(false);
});

test('SplitView performs a real split with live per-file progress', async (): Promise<void> => {
  // Regression guard for the fake-progress bug: builds a real
  // archive via @pendex/compile, splits through @pendex/split's
  // View, verifies files land byte-perfect.
  const state = {
    theme: FallbackTheme,
    config: testConfig,
  };
  await new Compile(state).execute();

  const view = new SplitView(state);
  expect(view.render()).resolves.toBeUndefined();

  const rebuilt: string = await Bun.file(join('src', 'index.ts')).text();
  expect(rebuilt).toBe('console.log("hello");\n');
});

test('Split handles a missing manifest without throwing', async (): Promise<void> => {
  const splitCmd = new Split({
    theme: FallbackTheme,
    config: testConfig,
  });

```

```

    });
    expect(splitCmd.execute()).resolves.toBeUndefined();
  });

  test('Split handles a missing archive file without throwing', async (): Promise<void> => {
    mkdirSync('OUT', { recursive: true });
    writeFileSync(
      'OUT/manifest.json',
      JSON.stringify({
        files: { 'missing.txt': [] },
        emptyDirectories: [],
      }),
    );
    const splitCmd = new Split({
      theme: FallbackTheme,
      config: testConfig,
    });
    expect(splitCmd.execute()).resolves.toBeUndefined();
  });

  test('empty directories recorded at compile are restored at split', async (): Promise<void> => {
    const state = {
      theme: FallbackTheme,
      config: testConfig,
    };
    await new Compile(state).execute();

    const manifest: Manifest = await Bun.file(
      join('OUT', 'manifest.json'),
    ).json();
    const match = manifest.emptyDirectories
      .map((d: string) => d.replace(/\\/g, '/'))
      .some((d: string) => /test-sandbox-\\d+\\src\\empty-dir\\.test(d));

    expect(match).toBe(true);

    await new Split(state).execute();

    const rebuiltEmptyDir: string = join('REBUILT', 'src', 'empty-dir');
    expect(existsSync(rebuiltEmptyDir)).toBe(false);
  });
});

// =====< END >-=====
// FILE NAME: : integration.test.ts
// TYPE      : .TS
// PATH      : packages/utils/tests/integration.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : validators.test.ts
// TYPE      : .TS
// PATH      : packages/utils/tests/validators.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
import { describe, expect, test } from 'bun:test';
import { validateDirectory, validateGlobInput } from '../src/utils/validators';

describe('Validators', () => {
  describe('validateDirectory', () => {
    test('should return error for empty input', () => {
      expect(validateDirectory('')).toBe(

```

```

        'This field is required and cannot be left blank.',
    );
    expect(validateDirectory('')).toBe(
        'This field is required and cannot be left blank.',
    );
});

test('should return error for input with spaces', () => {
    expect(validateDirectory('my folder')).toBe(
        'Spaces are not allowed in directory path names.',
    );
});

test('should return error for invalid characters', () => {
    expect(validateDirectory('folder<name')).toBe(
        'Directory name contains illegal path symbols (e.g., \\. , : , * , ? , " , < , > , | ).',
    );
    expect(validateDirectory('folder>name')).toBe(
        'Directory name contains illegal path symbols (e.g., \\. , : , * , ? , " , < , > , | ).',
    );
    expect(validateDirectory('folder:name')).toBe(
        'Directory name contains illegal path symbols (e.g., \\. , : , * , ? , " , < , > , | ).',
    );
    expect(validateDirectory('folder"name')).toBe(
        'Directory name contains illegal path symbols (e.g., \\. , : , * , ? , " , < , > , | ).',
    );
    expect(validateDirectory('folder|name')).toBe(
        'Directory name contains illegal path symbols (e.g., \\. , : , * , ? , " , < , > , | ).',
    );
    expect(validateDirectory('folder?name')).toBe(
        'Directory name contains illegal path symbols (e.g., \\. , : , * , ? , " , < , > , | ).',
    );
    expect(validateDirectory('folder*name')).toBe(
        'Directory name contains illegal path symbols (e.g., \\. , : , * , ? , " , < , > , | ).',
    );
});

test('should return undefined for valid input', () => {
    expect(validateDirectory('validfolder')).toBeUndefined();
    expect(validateDirectory('valid-folder')).toBeUndefined();
    expect(validateDirectory('valid_folder')).toBeUndefined();
});

describe('validateGlobInput', () => {
    test('should return error for empty input', () => {
        expect(validateGlobInput('')).toBe(
            'This field is required and cannot be left blank.',
        );
        expect(validateGlobInput('')).toBe(
            'This field is required and cannot be left blank.',
        );
    });

    test('should return error for empty glob patterns', () => {
        expect(validateGlobInput(' ', '')).toBe(
            'Glob lists cannot contain completely blank strings.',
        );
        expect(validateGlobInput('pattern,,pattern')).toBe(
            'Glob lists cannot contain completely blank strings.',
        );
        expect(validateGlobInput('pattern, ')).toBe(
            'Glob lists cannot contain completely blank strings.',
        );
    });
});

```



```

    );
  });

  test('should return undefined for valid input', () => {
    expect(validateGlobInput('**/*.ts')).toBeUndefined();
    expect(validateGlobInput('pattern1,pattern2')).toBeUndefined();
  });
});

});

// =====< END >-=====
// FILE NAME: : validators.test.ts
// TYPE      : .TS
// PATH      : packages/utils/tests/validators.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : index.test.ts
// TYPE      : .TS
// PATH      : packages/utils/tests/index.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// =====< START >-=====
// oxlint-disable typescript/no-explicit-any
import { describe, expect, spyOn, test, type Mock } from 'bun:test';
import { Message } from '../src';

describe('Entry Point (index.ts)', (): void => {
  test('Message function should handle Error objects', (): void => {
    const consoleSpy: Mock<any> = spyOn(
      console,
      'error',
    ).mockImplementation((): void => {});
    try {
      Message(new Error('Test error'));
      expect(consoleSpy).toHaveBeenCalledWith('Fatal crash: Test error');
    } finally {
      // Restore the original console.error function
      consoleSpy.mockRestore();
    }
  });
});

  test('Message function should handle non-Error messages', (): void => {
    const consoleSpy: Mock<any> = spyOn(
      console,
      'error',
    ).mockImplementation((): void => {});
    try {
      Message('A simple string error');
      expect(consoleSpy).toHaveBeenCalledWith(
        'Fatal crash: A simple string error',
      );
    } finally {
      consoleSpy.mockRestore();
    }
  });
});
});

// =====< END >-=====
// FILE NAME: : index.test.ts
// TYPE      : .TS
// PATH      : packages/utils/tests/index.test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z

```

```

//===== < END >=====

//===== < START >=====
// FILE NAME: : string-extension-test.ts
// TYPE      : .TS
// PATH      : utils/string-extension-test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
//===== < START >=====
// oxlint-disable typescript/no-explicit-any
import { errorNote } from './errorNote';

import { intro, log, outro } from '@clack/prompts';
import { Colors } from '@pendex/color';

import './string-extensions';
import type { PathSegment } from './string-extensions';

// DEBUG START

/**
 * Runs a suite of test cases to thoroughly validate the
 * custom String extension methods.
 * Logs detailed pass/fail results directly to the console.
 */
export const testPathExtensions = async (): Promise<void> => {
  interface TestCase {
    name: string;
    input: string;
    segments?: PathSegment[];
    expectedPosix: string;
    expectedNormalized: string;
  }

  const testCases: TestCase[] = [
    {
      name: 'Standard Windows Path',
      input: 'C:\\Users\\Admin\\Documents\\projects',
      expectedPosix: 'C:/Users/Admin/Documents/projects',
      expectedNormalized: 'C:/Users/Admin/Documents/projects',
    },
    {
      name: 'Mixed Slashes',
      input: 'src\\components\\buttons\\SubmitButton.tsx',
      expectedPosix: 'src/components/buttons/SubmitButton.tsx',
      expectedNormalized: 'src/components/buttons/SubmitButton.tsx',
    },
    {
      name: 'Consecutive Slashes (Internal)',
      input: 'folder/subfolder//nested///file.txt',
      expectedPosix: 'folder/subfolder//nested///file.txt',
      expectedNormalized: 'folder/subfolder/nested/file.txt',
    },
    {
      name: 'Windows UNC Network Share',
      input: '\\\\server\\share\\folder',
      expectedPosix: '//server/share/folder',
      expectedNormalized: '//server/share/folder',
    },
    {
      name: 'Web Protocol (HTTP)',
      input: 'http://example.com',
      expectedPosix: 'http://example.com',
      expectedNormalized: 'http://example.com',
    },
  ],
  {
    name: 'Web Protocol (HTTP)',
    input: 'http://example.com',
    expectedPosix: 'http://example.com',
    expectedNormalized: 'http://example.com',
  },
];

```

```

    },
    {
      name: 'Windows Drive Letter with Double Slash',
      input: 'C:\\\\folder\\\\subfolder',
      expectedPosix: 'C://folder//subfolder',
      expectedNormalized: 'C:/folder/subfolder',
    },
    {
      name: 'Trailing Slashes',
      input: 'usr/local/bin/',
      expectedPosix: 'usr/local/bin/',
      expectedNormalized: 'usr/local/bin/',
    },
    {
      name: 'joinPath: Standard Append',
      input: 'src',
      segments: ['components', 'Button.tsx'],
      expectedPosix: 'src/components/Button.tsx',
      expectedNormalized: 'src/components/Button.tsx',
    },
    {
      name: 'joinPath: Falsy Conditions & Numbers',
      input: 'src',
      // oxlint-disable-next-line no-constant-binary-expression
      segments: ['v1', false && 'premium', 'users', 404],
      expectedPosix: 'src/v1/users/404',
      expectedNormalized: 'src/v1/users/404',
    },
    {
      name: 'joinPath: Mixed Backslashes in Segments',
      input: 'src',
      segments: ['assets\\images', 'logo.png'],
      expectedPosix: 'src/assets/images/logo.png',
      expectedNormalized: 'src/assets/images/logo.png',
    },
  ],
  /**
   * PURPOSEFULLY FAILING TESTS
   */
  {
    name: 'Intentional Failure: Incorrect Mapped Path',
    input: 'C:\\Users\\Guest',
    expectedPosix: 'C:/Wrong/Path/Here',
    expectedNormalized: 'C:/Wrong/Path/Here',
  },
  {
    name: 'Intentional Failure: joinPath Output Mismatch',
    input: 'dist',
    segments: ['bundles', 'app.js'],
    expectedPosix: 'dist/wrong/app.js',
    expectedNormalized: 'dist/wrong/app.js',
  },
];

console.log();
intro(
  Colors.bgYellow(Colors.black(Colors.bold('STRING EXTENSIONS TESTING'))),
);

const deferredFailures: any[] = [];

// Process tasks sequentially to retain full control
// over failures without process abandonment
for (const [index, test] of testCases.entries()) {

```

```

const actualPosix = test.segments
  ? test.input.joinPath(...test.segments)
  : test.input.toPosixPath();

const actualNormalized = test.segments
  ? actualPosix
  : test.input.normalizePath();

const posixPassed = actualPosix === test.expectedPosix;
const normalizedPassed = actualNormalized === test.expectedNormalized;

const passedTag = Colors.green('Passed');
const failedTag = Colors.red('Failed');
const testNumber = Colors.cyan(`Test #${index + 1}`);
const testName = Colors.magenta(`${test.name}`);

if (!posixPassed || !normalizedPassed) {
  deferredFailures.push({
    name: test.name,
    input: test.input,
    segments: test.segments,
    posix: !posixPassed,
    normalized: !normalizedPassed,
    expectedPosix: test.expectedPosix,
    actualPosix,
    expectedNormalized: test.expectedNormalized,
    actualNormalized,
  });

  log.error(`${failedTag}...${testNumber}: ${testName}`);
} else {
  log.success(`${passedTag}...${testNumber}: ${testName}`);
}
}

const completed = () => {
  const complete = '✅ All path extension tests passed successfully!';
  outro(Colors.green(Colors.bold(complete)));
};

try {
  if (deferredFailures.length > 0) {
    throw new Error(JSON.stringify(deferredFailures));
  }
  completed();
} catch (error: any) {
  const errorTitle = 'Test Suite Completed with Failures: ';
  log.error(Colors.bgRed(Colors.black(Colors.bold(errorTitle))));

  let failuresToPrint: any[];

  try {
    failuresToPrint = JSON.parse(error.message);
  } catch {
    console.error(error);
    outro(Colors.red('Fatal infrastructure failure occurred.'));
    process.exit(1);
  }

  for (const details of failuresToPrint) {
    const lines: [string, string][] = [
      ['Input'.label(), `${details.input}`],
    ];

```

```

        if (details.segments) {
            lines.push([
                'Segments'.label(),
                JSON.stringify(details.segments),
            ]);
        }
        if (details.posix) {
            lines.push([
                'Expected Posix'.label(),
                `${details.expectedPosix}`,
            ]);
            lines.push([
                'Got Posix'.label(),
                `${details.actualPosix}`.red(),
            ]);
        }
        if (details.normalized) {
            lines.push([
                'Expected Normalized'.label(),
                `${details.expectedNormalized}`,
            ]);
            lines.push([
                'Got Normalized'.label(),
                `${details.actualNormalized}`.red(),
            ]);
        }
    }

    const maxLength = Math.max(...lines.map(([label]) => label.length));
    const noteContent = lines
        .map(
            ([label, value]) => `${label.padEnd(maxLength)} : ${value}`,
        )
        .join('\n');

    errorNote(
        noteContent,
        Colors.red(`Failure Details: ${details.name}`),
    );
}

log.error(Colors.red('Process terminated due to test failures.'));
}

completed();
};

// Execute the tests
export const runTests = async () => {
    try {
        await testPathExtensions();
        console.log('TESTING COMPLETE!!!');
        process.exit(0);
    } catch (err: any) {
        console.error(err);
        process.exit(1);
    }
    process.exit(0);
};

//runTests();

// DEBUG END

```

```

// ██████████< END >────────────────────────────────────────────────────────────────────────────────
// FILE NAME: : string-extension-test.ts
// TYPE      : .TS
// PATH      : utils/string-extension-test.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< END >────────────────────────────────────────────────────────────────────────────────

// ██████████< START >────────────────────────────────────────────────────────────────────────────────
// FILE NAME: : preload.ts
// TYPE      : .TS
// PATH      : tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< START >────────────────────────────────────────────────────────────────────────────────
// FILE-PATH: tests/preload.ts
/**
 * @file tests/preload.ts
 * @description Global preload for Bun test suite. Mocks UI dependencies and manages isolated
 * temporary sandbox environments with guaranteed post-suite cleanup.
 */

import type { Task } from '@clack/prompts';
import * as OriginalCore from '@pendex/core';
import { afterAll, mock, type Mock } from 'bun:test';
import { existsSync, mkdirSync, rmSync } from 'node:fs';
import { join } from 'node:path';

/**
 * Generic function signatures for Clack UI mock objects.
 */
type UnknownFunction = (...args: unknown[]) => unknown;

type SpinnerFunction = {
  start: Mock<UnknownFunction>;
  stop: Mock<UnknownFunction>;
  message: Mock<UnknownFunction>;
};

/**
 * Dynamic Package Resolution for Monorepos.
 * Resolves the target monorepo package root dynamically from `Bun.main`.
 */
const currentTestPath: string = Bun.main;

const packageMatch: RegExpMatchArray | null = currentTestPath.match(
  /[\\/]packages[\\/]([^\v]+)/,
);

const currentPackageName: string | null | undefined = packageMatch
  ? packageMatch[1]
  : null;

if (!currentPackageName) {
  throw new Error(
    `Testing framework could not resolve the package name from path: ${currentTestPath}`,
  );
}

/**
 * Dynamically construct the path to the current package's root directory.
 */
const PACKAGE_ROOT: string = join(
  process.cwd(),

```

```

    'packages',
    currentPackageName,
  );

  /**
   * Sandbox Directory Setup (isolated per process ID per package suite).
   */
  const UNIQUE_DIR: string = `test-sandbox-${process.pid}`;

  export const ABSOLUTE_SANDBOX_PATH: string = join(
    PACKAGE_ROOT,
    'tests',
    UNIQUE_DIR,
  );

  const RUNTIME_CONFIG_PATH: string = join(
    ABSOLUTE_SANDBOX_PATH,
    'runtime.config.json',
  );

  // Registry of dynamically created temporary sandbox paths during test runs.
  const activeSandboxPaths: Set<string> = new Set<string>([
    ABSOLUTE_SANDBOX_PATH,
  ]);

  if (!existsSync(ABSOLUTE_SANDBOX_PATH)) {
    mkdirSync(ABSOLUTE_SANDBOX_PATH, { recursive: true });
  }

  /**
   * Mock Core Runtime Config Path
   */
  void mock.module('@pendex/core', () => ({
    ...OriginalCore,
    Constants: {
      // oxlint-disable-next-line typescript/no-misused-spread
      ...OriginalCore.Constants,
      RUNTIME_CONFIG_PATH,
    },
  }));

  /**
   * Global Teardown Hook
   * Guarantees that the root process sandbox directory and all registered temporary directories
   * are forcibly removed after all test suites complete.
   */
  afterAll((): void => {
    for (const dirPath of activeSandboxPaths) {
      if (existsSync(dirPath)) {
        rmSync(dirPath, {
          recursive: true,
          force: true,
        });
      }
    }
  });

  /**
   * Global UI Mocking for Clack Prompts
   */
  void mock.module('@clack/prompts', () => ({
    intro: mock(),
    outro: mock(),
  }));

```



```
// TYPE      : .TS
// PATH      : packages/core/tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████ < START > ██████████
// FILE-PATH: packages/core/tests/preload.ts
//
// **
// * @file packages/core/tests/preload.ts
// * @description Package-isolated preload for @pendex/core. Mocks UI interactions
// * and handles cleanup for test sandboxes scoped strictly to packages/core/tests/.
// */

import type { Task } from '@clack/prompts';
import { afterAll, mock, type Mock } from 'bun:test';
import { existsSync, mkdirSync, rmSync } from 'node:fs';
import { isAbsolute, join, resolve } from 'node:path';

type MockFunction = (...args: unknown[]) => unknown;

type SpinnerMock = {
  start: Mock<MockFunction>;
  stop: Mock<MockFunction>;
  message: Mock<MockFunction>;
};

/**
 * Absolute path to this package's tests directory.
 * `import.meta.dir` is always absolute to this file (`packages/core/tests`).
 */
export const PACKAGE_TESTS_DIR: string = import.meta.dir;
const UNIQUE_DIR: string = `test-sandbox-${process.pid}`;

export const ABSOLUTE_SANDBOX_PATH: string = join(
  PACKAGE_TESTS_DIR,
  UNIQUE_DIR,
);

const activeSandboxPaths: Set<string> = new Set<string>();

const tasksMock = mock(async (tasks: Task[]): Promise<void> => {
  for (const t of tasks) {
    if (typeof t.task === 'function') {
      await t.task(mock());
    }
  }
});

/**
 * Global UI Mocking for @clack/prompts within @pendex/core
 */
void mock.module('@clack/prompts', () => ({
  intro: mock(),
  outro: mock(),
  box: mock(),
  note: mock(),
  text: mock(),
  select: mock(),
  confirm: mock(),
  multiselect: mock(),
  isCancel: (val: unknown): boolean => val === Symbol.for('cancel'),
  spinner: (): SpinnerMock => ({
    start: mock(),
    stop: mock(),
  })
}));
```

```

        message: mock(),
    }},
    tasks: tasksMock,
    log: {
        success: mock(),
        warn: mock(),
        error: mock(),
        info: mock(),
        step: mock(),
        message: mock(),
    },
});

/**
 * Resolves a given directory path safely against PACKAGE_TESTS_DIR if relative,
 * avoiding process.cwd() shifts.
 */
function resolveSandboxPath(dir: string): string {
    if (isAbsolute(dir)) {
        return dir;
    }
    return resolve(PACKAGE_TESTS_DIR, dir);
}

/**
 * Creates a package-scoped sandbox directory once per suite and registers it for cleanup.
 */
export async function createSandbox(dir: string): Promise<void> {
    const absoluteDir = resolveSandboxPath(dir);
    activeSandboxPaths.add(absoluteDir);

    if (existsSync(absoluteDir)) {
        rmSync(absoluteDir, { recursive: true, force: true });
    }
    mkdirSync(absoluteDir, { recursive: true });
}

/**
 * Removes a sandbox directory once per suite and unregisters it from cleanup.
 */
export async function cleanupSandbox(dir: string): Promise<void> {
    const absoluteDir = resolveSandboxPath(dir);

    if (existsSync(absoluteDir)) {
        rmSync(absoluteDir, { recursive: true, force: true });
    }
    activeSandboxPaths.delete(absoluteDir);
}

/**
 * Package Teardown Hook
 * Automatically cleans up all registered temporary sandboxes after all tests in this package complete.
 */
afterAll(() : void => {
    for (const dirPath of activeSandboxPaths) {
        if (existsSync(dirPath)) {
            try {
                rmSync(dirPath, { recursive: true, force: true });
            } catch {
                // Ignore lock errors during process exit
            }
        }
    }
}
}

```

```

});

// ██████████< END >- ██████████
// FILE NAME: : preload.ts
// TYPE      : .TS
// PATH      : packages/core/tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< END >- ██████████

// ██████████< START >- ██████████
// FILE NAME: : preload.ts
// TYPE      : .TS
// PATH      : packages/compile/tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< START >- ██████████
// FILE-PATH: packages/compile/tests/preload.ts
//
/**
 * @file packages/compile/tests/preload.ts
 * @description Package-isolated preload for @pendex/compile. Mocks UI interactions
 * and handles cleanup for test sandboxes scoped strictly to packages/compile/tests/.
 */
import type { Task } from '@clack/prompts';
import { afterAll, mock, type Mock } from 'bun:test';
import { existsSync, mkdirSync, rmSync } from 'node:fs';
import { isAbsolute, join, resolve } from 'node:path';

type MockFunction = (...args: unknown[]) => unknown;

type SpinnerMock = {
  start: Mock<MockFunction>;
  stop: Mock<MockFunction>;
  message: Mock<MockFunction>;
};

/**
 * Absolute path to this package's tests directory.
 * `import.meta.dir` is always absolute to this file (`packages/core/tests`).
 */
export const PACKAGE_TESTS_DIR: string = import.meta.dir;
const UNIQUE_DIR: string = `test-sandbox-${process.pid}`;

export const ABSOLUTE_SANDBOX_PATH: string = join(
  PACKAGE_TESTS_DIR,
  UNIQUE_DIR,
);

const activeSandboxPaths: Set<string> = new Set<string>();

/**
 * Global UI Mocking for @clack/prompts within @pendex/core
 */
void mock.module('@clack/prompts', () => ({
  intro: mock(),
  outro: mock(),
  box: mock(),
  note: mock(),
  text: mock(),
  select: mock(),
  confirm: mock(),
  multiselect: mock(),
  isCancel: (val: unknown): boolean => val === Symbol.for('cancel'),
  spinner: (): SpinnerMock => ({

```

```

        start: mock(),
        stop: mock(),
        message: mock(),
    }},
    tasks: mock(async (tasks: Task[]): Promise<void> => {
        for (const t of tasks) {
            if (typeof t.task === 'function') {
                await t.task(mock());
            }
        }
    })),
    log: {
        success: mock(),
        warn: mock(),
        error: mock(),
        info: mock(),
        step: mock(),
        message: mock(),
    },
}));

/**
 * Resolves a given directory path safely against PACKAGE_TESTS_DIR if relative,
 * avoiding process.cwd() shifts.
 */
function resolveSandboxPath(dir: string): string {
    if (isAbsolute(dir)) {
        return dir;
    }
    return resolve(PACKAGE_TESTS_DIR, dir);
}

/**
 * Creates a package-scoped sandbox directory once per suite and registers it for cleanup.
 */
export async function createSandbox(dir: string): Promise<void> {
    const absoluteDir = resolveSandboxPath(dir);
    activeSandboxPaths.add(absoluteDir);

    if (existsSync(absoluteDir)) {
        rmSync(absoluteDir, { recursive: true, force: true });
    }
    mkdirSync(absoluteDir, { recursive: true });
}

/**
 * Removes a sandbox directory once per suite and unregisters it from cleanup.
 */
export async function cleanupSandbox(dir: string): Promise<void> {
    const absoluteDir = resolveSandboxPath(dir);

    if (existsSync(absoluteDir)) {
        rmSync(absoluteDir, { recursive: true, force: true });
    }
    activeSandboxPaths.delete(absoluteDir);
}

/**
 * Package Teardown Hook
 * Automatically cleans up all registered temporary sandboxes after all tests in this package complete.
 */
afterAll(): void => {
    for (const dirPath of activeSandboxPaths) {

```

```

    if (existsSync(dirPath)) {
      try {
        rmSync(dirPath, { recursive: true, force: true });
      } catch {
        // Ignore lock errors during process exit
      }
    }
  }
}

});

// ██████████< END >-██████████
// FILE NAME: : preload.ts
// TYPE      : .TS
// PATH      : packages/compile/tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : preload.ts
// TYPE      : .TS
// PATH      : packages/exit/tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< START >-██████████
// FILE-PATH: packages/exit/tests/preload.ts
//

/**
 * @file packages/exit/tests/preload.ts
 * @description Package-isolated preload for @pendex/exit. Mocks UI interactions
 * for test suites scoped strictly to packages/exit/tests/.
 */

import type { Task } from '@clack/prompts';
import { mock } from 'bun:test';

type MockFunction = (...args: unknown[]) => unknown;

type SpinnerMock = {
  start: Mock<MockFunction>;
  stop: Mock<MockFunction>;
  message: Mock<MockFunction>;
};

/**
 * Global UI Mocking for @clack/prompts within @pendex/exit
 */
void mock.module('@clack/prompts', () => ({
  intro: mock(),
  outro: mock(),
  box: mock(),
  note: mock(),
  text: mock(),
  select: mock(),
  confirm: mock(),
  multiselect: mock(),
  isCancel: (val: unknown): boolean => val === Symbol.for('cancel'),
  spinner: (): SpinnerMock => ({
    start: mock(),
    stop: mock(),
    message: mock(),
  }),
  tasks: mock(async (tasks: Task[]): Promise<void> => {
    for (const t of tasks) {

```

```

        if (typeof t.task === 'function') {
            await t.task(mock());
        }
    },
    log: {
        success: mock(),
        warn: mock(),
        error: mock(),
        info: mock(),
        step: mock(),
        message: mock(),
    },
});

// ██████████< END >────────────────────────────────────────────────────────────────────────────────
// FILE NAME: : preload.ts
// TYPE      : .TS
// PATH      : packages/exit/tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< END >────────────────────────────────────────────────────────────────────────────────

// ██████████< START >────────────────────────────────────────────────────────────────────────────────
// FILE NAME: : preload.ts
// TYPE      : .TS
// PATH      : packages/split/tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< START >────────────────────────────────────────────────────────────────────────────────
// FILE-PATH: packages/split/tests/preload.ts
//
/**
 * @file packages/split/tests/preload.ts
 * @description Package-isolated preload for @pendex/split. Mocks UI interactions
 * and handles cleanup for test sandboxes scoped strictly to packages/split/tests/.
 */
/**
 * @file packages/compile/tests/preload.ts
 * @description Package-isolated preload for @pendex/compile. Mocks UI interactions
 * and handles cleanup for test sandboxes scoped strictly to packages/compile/tests/.
 */

import type { Task } from '@clack/prompts';
import { afterAll, mock, type Mock } from 'bun:test';
import { existsSync, mkdirSync, rmSync } from 'node:fs';
import { isAbsolute, join, resolve } from 'node:path';

type MockFunction = (...args: unknown[]) => unknown;

type SpinnerMock = {
    start: Mock<MockFunction>;
    stop: Mock<MockFunction>;
    message: Mock<MockFunction>;
};

/**
 * Absolute path to this package's tests directory.
 * `import.meta.dir` is always absolute to this file (`packages/core/tests`).
 */
export const PACKAGE_TESTS_DIR: string = import.meta.dir;
const UNIQUE_DIR: string = `test-sandbox-${process.pid}`;

export const ABSOLUTE_SANDBOX_PATH: string = join(
    PACKAGE_TESTS_DIR,
    UNIQUE_DIR,

```

```

);

const activeSandboxPaths: Set<string> = new Set<string>();

/**
 * Global UI Mocking for @clack/prompts within @pendex/core
 */
void mock.module('@clack/prompts', () => ({
  intro: mock(),
  outro: mock(),
  box: mock(),
  note: mock(),
  text: mock(),
  select: mock(),
  confirm: mock(),
  multiselect: mock(),
  isCancel: (val: unknown): boolean => val === Symbol.for('cancel'),
  spinner: (): SpinnerMock => ({
    start: mock(),
    stop: mock(),
    message: mock(),
  }),
  tasks: mock(async (tasks: Task[]): Promise<void> => {
    for (const t of tasks) {
      if (typeof t.task === 'function') {
        await t.task(mock());
      }
    }
  }),
  log: {
    success: mock(),
    warn: mock(),
    error: mock(),
    info: mock(),
    step: mock(),
    message: mock(),
  },
}));

/**
 * Resolves a given directory path safely against PACKAGE_TESTS_DIR if relative,
 * avoiding process.cwd() shifts.
 */
function resolveSandboxPath(dir: string): string {
  if (isAbsolute(dir)) {
    return dir;
  }
  return resolve(PACKAGE_TESTS_DIR, dir);
}

/**
 * Creates a package-scoped sandbox directory once per suite and registers it for cleanup.
 */
export async function createSandbox(dir: string): Promise<void> {
  const absoluteDir = resolveSandboxPath(dir);
  activeSandboxPaths.add(absoluteDir);

  if (existsSync(absoluteDir)) {
    rmSync(absoluteDir, { recursive: true, force: true });
  }
  mkdirSync(absoluteDir, { recursive: true });
}

```

```

/**
 * Removes a sandbox directory once per suite and unregisters it from cleanup.
 */
export async function cleanupSandbox(dir: string): Promise<void> {
    const absoluteDir = resolveSandboxPath(dir);

    if (existsSync(absoluteDir)) {
        rmSync(absoluteDir, { recursive: true, force: true });
    }
    activeSandboxPaths.delete(absoluteDir);
}

/**
 * Package Teardown Hook
 * Automatically cleans up all registered temporary sandboxes after all tests in this package complete.
 */
afterAll((): void => {
    for (const dirPath of activeSandboxPaths) {
        if (existsSync(dirPath)) {
            try {
                rmSync(dirPath, { recursive: true, force: true });
            } catch {
                // Ignore lock errors during process exit
            }
        }
    }
});

// ██████████< END >────────────────────────────────────────
// FILE NAME: : preload.ts
// TYPE      : .TS
// PATH      : packages/split/tests/preload.ts
// PROCESSED: 2026-09-06T21:55:36.433Z
// ██████████< END >────────────────────────────────────────

```